

COMSATS University Islamabad, Sahiwal Campus

FALCONS

(Flood Alert, Climate Observation & Navigation System for Safety)

By

Syed Haider Raza Zaidi CIIT/ FA22-BCS-153/SWL

Muhammad Hamza Farooq CIIT/ FA22-BCS-185/SWL

Muhammad Haris Khan CIIT/ FA22-BCS-186/SWL

Supervisor

Mr. Ali Sher Kashif

Bachelor of Science in Computer Science (2022-2026)

The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University Islamabad, Sahiwal Campus

FALCONS

(Flood Alert, Climate Observation & Navigation System for Safety)

**A project presented to
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment
Of the requirements for the degree of
Bachelor of Science in Computer Science (2022-2026)**

By

Syed Haider Raza Zaidi CIIT/ FA22-BCS-153/SWL

Muhammad Hamza Farooq CIIT/ FA22-BCS-185/SWL

Muhammad Haris Khan CIIT/ FA22-BCS-186/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied from any source. It is further declared that we have developed this software and the accompanying report entirely based on our personal efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted to any application for any other degree or qualification of this or any other university or institute of learning.

Syed Haider Raza Zaidi

Muhammad Hamza Farooq

Muhammad Haris Khan

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “FALCONS” was developed by **Syed Haider Raza Zaidi (CIIT/FA22-BCS-153/SWL)**, **Muhammad Hamza Farooq (CIIT/FA22-BCS-185/SWL)**, **Muhammad Haris Khan Farooq (CIIT/FA22-BCS- 186/SWL)** under the supervision of “**Mr. Ali Sher Kashif**” and co- supervisor “**Dr. Fakhar Mustafa**” and that in their opinion, it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor
Mr. Ali Sher Kashif
Lecturer
Department of Computer Science

Internal Examiner

Dr Zafar Iqbal Roy
Assistant Professor
Head of Department
Department of Computer Science
COMSATS University Islamabad, Sahiwal campus

Executive Summary

Floods are among the most devastating natural disasters, causing significant loss of life, infrastructure damage, and economic instability, particularly in developing countries such as Pakistan. The increasing intensity of climate change and unpredictable weather patterns has further highlighted the need for efficient early warning and disaster management systems. However, existing solutions are often fragmented, lack predictive capabilities, and fail to provide real-time, user-friendly communication for the public.

To address these challenges, this project proposes ***FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)***, an integrated web-based platform designed to enhance disaster preparedness, prediction, and response. The system utilizes machine learning techniques to analyze environmental parameters such as rainfall, temperature, humidity, and wind speed to predict flood risk levels. In addition, **FALCONS** incorporates **real-time weather data, interactive flood mapping, tourism safety advisories, and an alert notification system** to provide comprehensive situational awareness.

The platform also enables users to report emergency situations through a rescue reporting module, allowing for improved coordination between the public and response authorities. An administrative interface is included to manage data, monitor conditions, and broadcast alerts effectively. By combining predictive analytics with real-time information and user interaction, **FALCONS** aims to reduce disaster impact, improve decision-making, and enhance public safety.

This project demonstrates the potential of integrating **modern web technologies and machine learning** in developing smart disaster management solutions that can contribute to saving lives and minimizing damage in high-risk regions.

Acknowledgement

All praise is to Almighty Allah, who bestowed upon us a minute portion of His boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor, “**Ali Sher Kashif**”, and our co-supervisor, “**Fakhar Mustafa**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Syed Haider Raza Zaidi

Muhammad Hamza Farooq

Muhammad Haris Khan

Abbreviations

Abbreviation	Meaning
FALCONS	Flood Alert, Climate Observation & Navigation System for Safety
ML	Machine Learning
API	Application Programming Interface
GIS	Geographic Information System
UI	User Interface
UX	User Experience
DB	Database
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
PDMA	Provincial Disaster Management Authority
NDMA	National Disaster Management Authority
WMO	World Meteorological Organization
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
IoT	Internet of Things
AI	Artificial Intelligence
GPS	Global Positioning System
UI/UX	User Interface/User Experience

Table of Contents

1. Introduction	2
1.1 Brief.....	2
1.1.1 User Registration and Profile Management.....	3
1.1.2 AI-Based Flood Prediction and Climate Analysis.....	3
1.1.3 Flood Map and Smart Navigation	4
1.1.4 Alert System and Emergency Notifications	4
1.1.5 Rescue Reporting and Emergency Communication	5
1.1.6 Admin Panel and Monitoring System	5
1.1.7 FAQs	5
1.2 Relevance to Course Modules	6
1.2.1 Web Development.....	6
1.2.2 Software Engineering	6
1.2.3 Database Management Systems	7
1.2.4 Artificial Intelligence (AI).....	7
1.2.5 Human-Computer Interaction (HCI)	7
1.3 Project Background	8
1.4 Literature Review	8
1.5 Analysis from Literature Review	9
1.5.1 User Registration and Profile Management.....	9
1.5.2 Flood Prediction System	9
1.5.3 Mapping and Navigation System	10
1.5.4 Alert and Notification Module	10
1.5.5 Rescue Reporting and Admin Monitoring.....	10

1.6	Methodology and Software Lifecycle.....	10
1.6.1	Modular Development Approach	10
1.6.2	Object-Oriented Development (OOD).....	10
1.6.3	Incremental Approach	10
1.6.4	Rational Behind the Selected Methodology	11
1.6.5	Software Development Life Cycle (SDLC)	11
2	Problem Definition	15
2.1	Problem Statement.....	15
2.2	Deliverables Requirements.....	15
2.2.1	FALCONS System Platform	15
2.2.2	Documentation	16
2.2.3	Testing Reports	16
2.2.4	Deployment Setup	17
2.3	Development Requirements	17
2.3.1	Technologies and Tools.....	17
2.3.2	Server and Hosting Requirements	18
2.3.3	Performance and Scalability.....	18
2.3.4	Security and Privacy.....	19
2.3.5	Development Milestones	19
2.3.6	Collaboration and Version Control	19
2.4	Current System	20
3	Requirement Analysis	22
3.1	Use Case	22
3.1.1	Use Case Description	23
3.1.1.2	Normal flow	25
3.2	Functional Requirements.....	27

3.2.1	User's Module	27
3.2.2	Admin modules	27
3.3	Non-functional requirement	27
3.3.1	Performance	28
3.3.2	Security	28
3.3.3	Portability	28
3.3.5	Reliability.....	28
4	Design and Architecture	30
4.1	System Architecture	30
4.2	Data Representation.....	32
4.3	Process Flow/Representation.....	34
4.3.1	Activity Diagrams	34
4.3.2	Admin Activity Diagram.....	34
4.4	Process Flow.....	36
4.5	Design Models.....	38
4.5.1	Sequence Diagram.....	38
4.5.2	Class Diagram	40
4.6	Data Design	42
4.7	Data Dictionary	43
5	Implementation	45
5.1	Algorithms.....	45
5.1.1	User Authentication.....	45
5.1.2	Flood Prediction Algorithm	46
5.1.3	Alert Generation System	46
5.1.4	Admin Management & User Management.....	47
5.1.5	Rescue Reporting System.....	47
5.2	External APIs	48

5.3	User Interface	49
5.3.1	Home Screen	49
5.3.2	Dashboard Screen.....	50
5.3.3	Weather overview screen	51
5.3.4	Flood prediction	52
5.3.5	Tourism Safety Screen	53
5.3.6	Alert Management Screen.....	54
5.3.7	Rescue Reporting Screen	55
5.3.8	Login screen	56
5.3.9	Create account Screen	57
6	Testing and Evaluation.....	59
6.1	Manual Testing.....	59
6.1.1	Unit Testing.....	61
6.2	Functional Testing.....	62
6.3	Integration Testing	63
7	Conclusion.....	65
7.1	Future Work	65
7.1.1	Enhanced AI-Based Try-On Accuracy.....	65
7.1.2	Smart Alert System.....	66
7.1.3	Mobile Application Development	66
7.1.4	Integration with IoT and Sensors	66
7.1.5	Advanced Data Visualization	66
7.1.6	Admin Analytics and Monitoring.....	67
7.1.7	Multi-Language and Accessibility Support.....	67
7.1.8	Tourism Safety Enhancements	67

8. References..... 69

List of Figures

Figure 3.1-1 Use Case	19
Figure 4.1-1 System architecture	27
Figure 4.2-1 Data representation	29
Figure 4.3-1 Admin activity	31
Figure 4.4-1 Process flow	33
Figure 4.5-1 Sequence Diagram	35
Figure 4.5-2 Class diagram	37
Figure 5.3-1 home page	45
Figure 5.3-2 Dashboard page	46
Figure 5.3-3 Weather Overview	47
Figure 5.3-4 Flood Prediction	48
Figure 5.3-5 Tourism Page	49
Figure 5.3-6 Alert Management	50
Figure 5.3-7 Rescue Reporting	51
Figure 5.3-8 Login Screen	52
Figure 5.3-9 Create Account	53
Figure 5.3-10 Admin Panel	54

List of Tables

Table 1-1 literature view.....	6
Table 3-1 complete platform interaction.....	20
Table 3-2 normal flow	21
Table 3-3 Admin flow	22
Table 5-1 External API.....	44
Table 6-1: Admin Login.....	57

Chapter 1

INTRODUCTION

1. Introduction

In recent years, the increasing frequency and intensity of natural disasters have raised serious concerns about public safety, infrastructure resilience, and disaster preparedness. Among these disasters, floods are particularly devastating due to their sudden occurrence and widespread impact on both urban and rural communities. Countries like Pakistan are especially vulnerable due to seasonal monsoon rains, glacier melting, and inadequate drainage systems. These factors often result in large-scale destruction, population displacement, and economic losses [1].

Despite advancements in technology, there remains a significant gap in the availability of integrated systems that can provide early warnings, real-time environmental monitoring, and efficient communication between the public and emergency response authorities. Many existing solutions focus on isolated functionalities such as weather forecasting or alert notifications but fail to offer a unified platform that combines prediction, monitoring, navigation, and reporting [2].

To address these limitations, this project introduces **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)**, a comprehensive web-based system designed to enhance disaster awareness, prediction, and response. The system integrates machine learning, real-time data APIs, and interactive visualization tools to provide users with accurate flood risk assessments, climate observations, and navigation assistance [5].

The primary aim of FALCONS is not only to predict potential flood risks but also to empower users with actionable information and tools that can help minimize damage and ensure safety. By providing a centralized platform for monitoring, alerts, and emergency communication, the system contributes to improved disaster management and public awareness.

1.1 Brief

Floods are one of the most destructive natural disasters, especially in regions like Pakistan, where monsoon rains, river overflow, and climate change. This project introduces **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)**, a smart web-based platform designed to provide flood prediction, climate monitoring, navigation assistance, and emergency communication in a unified system. The system is divided into the following core modules:

1.1.1 User Registration and Profile Management

The user registration and profile management module plays a crucial role in providing personalized services within the FALCONS system. It allows users to create secure accounts by entering essential details such as name, email address, and password. This process ensures that each user has a unique identity within the system, enabling access to customized features and services.

Once registered, users can log in to their accounts and manage their profiles according to their preferences. This includes updating personal information, selecting preferred locations, and managing alert subscriptions. The system ensures data security through proper validation and authentication mechanisms, protecting user information from unauthorized access [4].

This module also enhances user experience by enabling location-based services. For example, users can receive alerts and predictions specific to their selected cities or regions. Additionally, storing user preferences helps in delivering targeted notifications and improving system efficiency. Overall, this component forms the foundation for user interaction within the platform and ensures a seamless and personalized experience [3].

1.1.2 AI-Based Flood Prediction and Climate Analysis

The AI-based flood prediction module is one of the core components of the FALCONS system, designed to provide accurate and reliable assessments of flood risk. This module utilizes machine learning algorithms trained on historical environmental data, including rainfall levels, temperature variations, humidity, and wind speed. By analyzing these parameters, the system can estimate the likelihood of flooding in each area [4].

The predictive model is built using techniques such as classification algorithms, which categorize risk levels into low, moderate, or high. This allows users to easily understand the severity of potential flood conditions. The system also updates predictions dynamically as new data becomes available, ensuring that users receive the most current and relevant information.

In addition to prediction, this module provides insights into climate patterns and trends. By presenting data visually through charts and graphs, users can better understand environmental changes over time. This not only improves awareness but also supports decision-making, allowing individuals and authorities to take preventive measures before a disaster occurs.

1.1.3 Flood Map and Smart Navigation

The flood map and navigation module provides users with a visual representation of flood-prone areas using geographic data. This feature is implemented using mapping technologies that display regions based on their risk levels through color-coded indicators. High-risk areas are highlighted clearly, allowing users to quickly identify dangerous zones [2].

In addition to visualization, the system offers navigation support by guiding users toward safer routes. This is particularly useful during emergencies when individuals need to avoid flooded roads or hazardous regions. The integration of mapping services enhances the practicality of the system and makes it more interactive.

The module also benefits tourists and travelers who may not be familiar with local conditions. By providing real-time hazard information, it helps users plan their journeys more safely. Overall, this feature improves situational awareness and contributes significantly to user safety [4].

1.1.4 Alert System and Emergency Notifications

The alert system is designed to provide timely notifications to users regarding potential risks and environmental changes. It plays a critical role in disaster management by ensuring that individuals receive warnings before a situation becomes dangerous. Alerts are generated based on both real-time data and predictive analysis, making them reliable and proactive [4].

Users can subscribe to alerts for specific locations, allowing them to receive personalized notifications. These alerts may include flood warnings, heavy rainfall notifications, or changes in environmental conditions. The system ensures that notifications are delivered promptly, enabling users to take immediate action.

This module enhances public safety by improving awareness and preparedness. By receiving early warnings, users can evacuate areas, secure their belongings, and avoid unnecessary risks. The alert system is therefore a key component in minimizing the impact of disasters.

This module enhances public safety by improving awareness and preparedness. By receiving early warnings, users can evacuate areas, secure their belongings, and avoid unnecessary risks. The alert system is therefore a key component in minimizing the impact of disasters [9].

1.1.5 Rescue Reporting and Emergency Communication

The rescue reporting module allows users to communicate emergency situations directly through the platform. In times of disaster, quick communication is essential for saving lives and coordinating rescue efforts. This feature enables users to submit detailed reports, including location, description, and severity of the situation.

These reports are instantly available to administrators and emergency response teams, allowing for faster decision-making and action. The system ensures that all reports are properly recorded and tracked, enabling authorities to monitor ongoing situations effectively.

This module also promotes community involvement, as users can actively participate in reporting incidents and helping others. Bridging the communication gap between the public and rescue teams, it significantly improves the efficiency of emergency response operations.

1.1.6 Admin Panel and Monitoring System

The admin panel serves as the control center of the FALCONS system, providing administrators with tools to manage and monitor all system activities. Through this interface, administrators can view user data, manage rescue reports, update environmental data, and broadcast alerts.

The monitoring capabilities of the admin panel allow authorities to track real-time conditions and identify high-risk areas. This helps in making informed decisions and deploying resources effectively during emergencies. The system also provides analytical insights, enabling administrators to evaluate performance and improve operations.

1.1.7 FAQs

The Frequently Asked Questions (FAQs) section is designed to assist users in understanding how to use the system effectively. It provides answers to common queries related to registration, prediction, alerts, and other features.

This section improves user experience by reducing confusion and providing quick guidance without the need for external support. It is especially helpful for new users who may not be familiar with the system.

1.2 Relevance to Course Modules

The development of the FALCONS system is closely aligned with multiple academic course modules, demonstrating the practical application of theoretical concepts learned throughout the degree program. This project integrates knowledge from software engineering, web development, database systems, artificial intelligence, and human-computer interaction to create a comprehensive and functional solution. By combining these disciplines, FALCONS not only addresses a real-world problem but also reflects the interdisciplinary nature of modern software development [8].

1.2.1 Web Development

Web development plays a fundamental role in the creation of the FALCONS platform, as the system is designed as a web-based application accessible to users through browsers. Technologies such as HTML, CSS, and JavaScript are used to build the user interface and ensure a responsive and visually appealing design. The front end is structured to provide easy navigation and a smooth user experience, allowing users to interact with different modules efficiently [3].

Modern design principles are applied to ensure that the interface is intuitive and accessible across different devices, including desktops, tablets, and mobile phones. The use of dynamic scripting enhances interactivity, enabling real-time updates and data visualization. Overall, the web development component ensures that the system is user-friendly, efficient, and widely accessible.

1.2.2 Software Engineering

Software engineering principles are applied throughout the development of FALCONS to ensure a structured and systematic approach. This includes requirement analysis, system design, implementation, testing, and documentation. The project follows best practices such as modular design, code reusability, and maintainability.

By dividing the system into independent modules, the development process becomes more organized and manageable. Each module can be developed and tested separately, reducing complexity and improving reliability. Additionally, software engineering practices ensure that the system meets quality standards and can be easily updated or extended in the future [2].

1.2.3 Database Management Systems

Database management is an essential component of the FALCONS system, as it involves storing and managing user data, environmental data, alerts, and reports. A structured database design ensures efficient data retrieval and storage, supporting the overall functionality of the system.

The database is used to maintain user profiles, track rescue reports, and store historical data for analysis. Proper normalization techniques are applied to avoid redundancy and maintain data integrity. The system also ensures secure handling of sensitive information, protecting user data from unauthorized access. Overall, the database component enhances the reliability and efficiency of the platform [4]

1.2.4 Artificial Intelligence (AI)

Artificial Intelligence is a key feature of the FALCONS system, particularly in the flood prediction module. Machine learning algorithms are used to analyze environmental data and generate predictions about flood risks. This allows the system to provide proactive insights rather than relying solely on historical trends.

The use of AI improves accuracy and enables the system to adapt to changing conditions. By continuously learning from data, the model can provide more reliable predictions over time. This integration of AI demonstrates the practical application of advanced computing techniques in solving real-world problems.

1.2.5 Human-Computer Interaction (HCI)

Human-Computer Interaction principles are applied to ensure that the system is easy to use and accessible to a wide range of users. The interface is designed with simplicity and clarity in mind, allowing users to navigate the system without difficulty.

Elements such as clear menus, readable text, and intuitive controls enhance usability. The system also considers accessibility by ensuring compatibility with different devices and screen sizes. By focusing on user experience, FALCONS ensures that its features are effectively utilized by both technical and non-technical users [9].

1.3 Project Background

Flood management has traditionally relied on manual observation and delayed communication, which often results in ineffective responses and increased damage. In many cases, people receive warnings too late, leaving little time for preparation or evacuation. This highlights the need for systems that can provide early warnings and real-time information [6].

With the advancement of technology, several solutions have been developed to address these challenges. However, most of these systems focus on individual aspects such as weather forecasting or alert notifications, without providing a comprehensive solution. Additionally, many systems lack user-friendly interfaces, making them difficult to use for the general public [10].

1.4 Literature Review

Table 1.1 summarizes The field of disaster management has seen significant advancements with the introduction of technology-based solutions. Various studies have explored the use of machine learning algorithms for flood prediction, demonstrating their effectiveness in analyzing large datasets and identifying patterns. These models are capable of providing accurate predictions based on environmental parameters, making them valuable tools for disaster preparedness.

In addition to prediction systems, early warning systems have been developed to provide alerts to the public. These systems use sensors and communication technologies to detect changes in environmental conditions and notify users. However, many of these systems are limited in scope and do not provide additional features such as mapping or navigation.

Table 1-1 literature view

Platform	Weakness	Proposed Solution in Optical Facial Fit
Weather Forecast Websites	Provide only raw weather data; no flood prediction	Integrates ML-based flood prediction with weather data

Government Alert Systems	Delayed or generic alerts; no personalization	Real-time, location-based alert system
GIS Flood Mapping Tools	Complex interface; not user-friendly	Simple, interactive flood maps with easy navigation
Disaster Management Apps	Limited features; lack of integration	All-in-one platform (prediction + alerts + maps + reporting)
Manual Rescue Systems	Slow communication and coordination	Real-time rescue reporting and admin monitoring

1.5 Analysis from Literature Review

The analysis highlights several gaps in existing disaster management systems. While many platforms provide useful features such as weather updates or alert notifications, they often operate independently and lack integration. This fragmentation makes it difficult for users to access all necessary information in one place, especially during emergencies.

Existing systems also suffer from limitations such as lack of real-time prediction, absence of user-friendly interfaces, and inefficient communication between users and authorities. Many platforms provide data but do not offer actionable insights, which reduces their effectiveness in real-world scenarios.

1.5.1 User Registration and Profile Management

Most existing disaster management systems do not provide personalized user accounts, limiting their ability to offer customized services. In contrast, the FALCONS system includes a user registration and profile management module that allows users to create accounts and personalize their experience.

1.5.2 Flood Prediction System

Many existing platforms rely on static data and do not provide predictive capabilities. The FALCONS system addresses this limitation by implementing a machine learning-based flood prediction module. This predictive capability provides users with early warnings, enabling them

1.5.3 Mapping and Navigation System

Traditional mapping systems are often complex and lack real-time updates. The FALCONS system enhances this functionality by providing interactive flood maps that highlight high-risk areas. This makes the mapping module more practical and user-friendly compared to existing solutions.

1.5.4 Alert and Notification Module

Existing alert systems often provide generic notifications that are not tailored to specific users or locations. The FALCONS system improves this by offering real-time, location-based alerts.

1.5.5 Rescue Reporting and Admin Monitoring

One of the major limitations of existing systems is the lack of efficient communication between users and rescue teams. The FALCONS system addresses this issue through a dedicated rescue reporting module. Users can report emergencies directly through the platform, providing details such as location and situation.

1.6 Methodology and Software Lifecycle

The development of FALCONS followed a modular, object-oriented, and incremental approach to ensure clarity, scalability, and efficiency.

1.6.1 Modular Development Approach

The FALCONS system is developed using a modular approach, where the system is divided into smaller components such as prediction, alerts, mapping, and reporting.

1.6.2 Object-Oriented Development (OOD)

Object-oriented principles are used to design the system, ensuring that components are reusable and maintainable. Classes and objects are used to represent different entities within the system, improving organization and scalability.

1.6.3 Incremental Approach

The system is developed incrementally, with each phase adding new features and improvements. This approach allows for continuous testing and refinement, ensuring that the final product is reliable and efficient.

1.6.4 Rational Behind the Selected Methodology

The selected methodology provides flexibility and adaptability, allowing the system to evolve based on requirements. It also ensures that errors can be identified and corrected early in the development

1.6.5 Software Development Life Cycle (SDLC)

For the development of the **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)** platform, the **Incremental Model** was adopted within the Software Development Life Cycle (SDLC). This model divides the overall system into smaller, manageable components known as increments, where each increment delivers a functional part of the system [4].

The selection of the incremental model was based on the nature of the FALCONS system, which consists of multiple independent yet interconnected modules such as flood prediction, weather monitoring, mapping, alert systems, and rescue reporting.

1.6.5.1 Planning and Requirements Gathering

In the initial phase, detailed requirements were gathered based on the primary objective of developing a comprehensive disaster management system. The focus was on understanding the needs of users, including the general public, travelers, and administrative authorities. The following key features were identified:

- Flood prediction system using environmental data and machine learning
- Weather monitoring services providing real-time updates
- Flood mapping for identifying high-risk areas
- Alert system for notifying users about potential hazards
- Rescue reporting module for emergency communication
- Tourism advisory system for safe travel guidance
- Admin dashboard for system monitoring and management

1.6.5.1.1 Design

Based A detailed object-oriented design was created for each module:

Frontend Design: A clean and responsive user interface using modern web technologies such as HTML, CSS, and JavaScript.

Backend Structure: Modeled using OOP concepts such as a User, Prediction Engine, Weather Module, Alert System, and Admin Panel.

Database Design: Normalized PostgreSQL schema using Supabase for relational data mapping.

AI Module: Designed for flood prediction and travel advisory

1.6.5.1.2 Implementation

Each Implementation followed an incremental rollout:

Initial development focused on core features such as user interface and navigation.

The flood prediction module was implemented using dataset processing and model integration.

Each module was developed as a separate component, with testing and integration following completion.

1.6.5.1.3 Testing

After Multiple testing layers ensured the reliability of FALCONS:

Unit Testing: Individual components such as flood prediction logic, data input handling, and user interactions were tested to ensure correctness.

Integration Testing: Different modules were tested together to ensure smooth communication between components such as the frontend, prediction module, and APIs.

User Acceptance Testing (UAT): Real users tested the virtual try-on experience, navigation, and

1.6.5.1.3 Deployment and Maintenance

After successful testing, the system was deployed in a live environment. Deployment involved configuring the hosting environment, integrating APIs, and ensuring that all modules function correctly.

- The front end was hosted on a web server for public access
- APIs were connected for real-time weather data
- Dataset integration was ensured for prediction functionality
- System configurations were optimized for performance

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

Floods remain one of the most critical natural disasters affecting human life, infrastructure, and economic stability, particularly in countries like Pakistan. Despite the availability of weather forecasting tools and disaster alerts, there is still a major gap in providing a unified system that combines prediction, monitoring, navigation, and emergency communication. The FALCONS system is designed to address these issues by providing a comprehensive platform that integrates flood prediction, climate observation, navigation support, alert systems, and rescue reporting [6].

2.1 Problem Statement

The core problem addressed in this project is the lack of a centralized, intelligent, and user-friendly disaster management system that can effectively predict flood risks, monitor environmental conditions, and facilitate communication during emergencies. Existing systems are either too technical, limited in functionality, or not accessible to the public. This limits their ability to provide early warnings. Furthermore, users often need to access multiple sources to gather information such as weather updates, flood maps, and emergency contacts, which is inefficient and time-consuming [7].

Another major issue is the lack of real-time communication tools that allow users to report emergencies directly to authorities. Therefore, there is a need for an integrated system like FALCONS that combines all essential features into a single, accessible platform.

2.2 Deliverables Requirements

2.2.1 FALCONS System Platform

This is the main software product: a **web-based disaster management platform** developed using modern web technologies and data-driven approaches. The platform offers:

Flood Prediction Module: Users can check flood risk by selecting their city.

Weather Monitoring Services: Real-time weather data is displayed, including temperature, humidity, and rainfall conditions.

Alert and Notification System: Users receive real-time alerts based on their selected location.

Rescue Reporting Module: Users can report emergencies directly through the system by submitting their location and situation details.

Tourism Advisory Module: Provides safety guidelines and travel recommendations for users visiting different areas, helping them avoid high-risk locations.

User Accounts: Users can register, log in, and manage their preferences, such as selected city and alert settings.

Admin Panel: Administrators can monitor system data, manage users, handle rescue reports, and send alerts.

2.2.2 Documentation

User Documentation: This documentation is designed for end-users and provides step-by-step guidance on how to use the system. It explains how to register, select a city, check flood predictions, view weather data, explore maps, and report emergencies. The documentation also includes instructions for receiving alerts and understanding risk indicators.

Technical Documentation: This documentation is intended for developers and technical stakeholders. It describes the overall system architecture, including frontend design, data processing logic, and integration with APIs and datasets. It also includes database schema details (Supabase), module interactions, and configuration steps for deployment.

Admin Guide: The admin guide provides detailed instructions for administrators to manage the system. It includes steps for logging into the admin panel, monitoring predictions, handling rescue reports, sending alerts, and maintaining system performance. It also explains how to analyze user activity and system logs.

2.2.3 Testing Reports

This section includes detailed documentation of all testing activities performed during the development of the FALCONS system. Testing ensures that the system is reliable, accurate, and user-friendly. Each test case includes inputs, expected outputs, and actual results, ensuring that all functionalities meet system requirements.

2.2.4 Deployment Setup

The deployment setup describes the process of making the FALCONS system available to users in a live environment. The system is deployed as a web-based application using a hosting platform. The deployment process includes:

- Configuring the frontend on a web server
- Connecting APIs for real-time weather data
- Integrating the dataset for flood prediction
- Setting up the **Supabase database** for data storage
- Managing environment variables and system configurations

2.3 Development Requirements

2.3.1 Technologies and Tools

2.4.1.2 Frontend Frontend

The frontend of the FALCONS system is developed using HTML, CSS, and JavaScript to create a responsive and user-friendly interface. The design focuses on simplicity and accessibility so that users from different backgrounds can easily interact with the system. Responsive design techniques are applied to ensure compatibility across desktops, tablets, and mobile devices.

To enhance interactivity, JavaScript is used for handling dynamic user inputs such as city selection, flood prediction display, and navigation between modules. The interface also includes structured layouts such as dashboards and navigation drawers, which improve usability and overall user experience.

2.4.1.2 Backend

The FALCONS system utilizes Supabase as a backend-as-a-service solution, eliminating the need for a traditional server-side framework. Supabase provides a PostgreSQL-based relational database along with auto-generated APIs and real-time capabilities, which significantly simplifies backend development.

and rescue reports, is stored in a structured and normalized database. The platform also integrates external APIs to retrieve real-time weather data, which is combined with stored datasets to generate accurate flood predictions. Secure communication between the frontend and backend is ensured through API-based interactions, along with proper validation of user inputs to maintain data integrity.

2.3.2 Server and Hosting Requirements

The FALCONS system is deployed on a reliable hosting environment that ensures consistent availability and performance. The hosting setup is configured to support concurrent users and real-time operations, which is essential for a disaster management platform.

The system domain is properly configured and linked with the hosting server, enabling easy access for users. Secure communication is maintained through SSL encryption, ensuring that all data transmitted between users and the platform remains protected. The hosting infrastructure is also designed to be scalable, allowing the system to accommodate increasing user traffic without performance issues.

2.3.3 Performance and Scalability

Performance is a critical requirement for the FALCONS system, as it must process real-time data and deliver results without delay. The system is optimized using efficient coding practices and streamlined data processing techniques to ensure fast response times.

Real-time API integration is carefully managed to reduce latency, while caching mechanisms are applied to minimize repeated data requests. The system is also designed with scalability in mind, allowing it to handle a large number of users during emergency situations. The use of cloud-based services such as Supabase further enhances scalability by providing flexible resource management and high availability.

2.3.4 Security and Privacy

Security and privacy are essential aspects of the FALCONS system due to the sensitive nature of user data and disaster-related information. The system implements secure authentication mechanisms to ensure that only authorized users can access specific features.

Role-Based Access Control (RBAC) is used to differentiate between user and administrative privileges, ensuring controlled access to system functionalities. Sensitive data is protected using encryption techniques, and all inputs are validated to prevent security vulnerabilities such as injection attacks. The platform follows best practices for data protection, ensuring that user information is handled responsibly and securely.

2.3.5 Development Milestones

The development process of the FALCONS system is structured into multiple phases to ensure systematic progress. The initial phase focuses on requirement analysis and system design, followed by the development of core functionalities such as user interface and navigation.

Subsequent phases involve the integration of flood prediction models, weather APIs, and advanced features such as mapping and alert systems. The final stages include testing, debugging, and deployment. This phased approach allows continuous improvement and ensures that each component of the system is properly validated before integration.

2.3.6 Collaboration and Version Control

Effective collaboration and version control are essential for managing the development process. Version control systems such as Git are used to track changes in the codebase and maintain different versions of the project.

These tools enable multiple developers to work on the system simultaneously while ensuring consistency and stability. Changes can be reviewed, managed, and reverted if necessary, reducing the risk of errors and improving overall project reliability.

2.4 Current System

The current landscape of disaster management systems is characterized by fragmentation and limited integration. Existing platforms typically provide isolated functionalities such as weather forecasting, alert notifications, or mapping services. However, these systems often fail to deliver a comprehensive solution that combines all essential features into a single platform.

Weather applications primarily focus on displaying environmental data without offering predictive insights. Similarly, alert systems tend to provide generic notifications that are not tailored to specific user locations, reducing their effectiveness. Mapping tools, while useful, are often complex and not easily accessible to non-technical users [4].

Another major limitation of current systems is the lack of efficient communication between users and authorities. Emergency reporting is often delayed due to the absence of real-time interaction mechanisms. Furthermore, many platforms do not provide centralized control for administrators, making it difficult to manage data and respond to critical situations effectively.

The FALCONS system addresses these shortcomings by integrating multiple functionalities into a unified platform. It combines flood prediction, weather monitoring, mapping, alert systems, and rescue reporting into a cohesive solution. This integrated approach enhances usability [10].

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The requirement analysis phase is one of the most critical stages in the development of the FALCONS system, as it defines the foundation upon which the entire project is built. This phase focuses on identifying user needs, understanding system expectations, and translating them into clearly defined functional and non-functional requirements. A well-structured requirement analysis ensures that the system developed is aligned with real-world needs and can solve the identified problem effectively [4].

For the FALCONS (Flood Alert, Climate Observation & Navigation System for Safety) The primary stakeholders include general users, travelers, and administrative authorities who interact with the system in different ways. Each stakeholder has unique requirements, such as accessing flood predictions, receiving alerts, or managing system data. Understanding these diverse needs is essential to designing a system that is both comprehensive and user-friendly [7].

3.1 Use Case

The case model of the FALCONS system provides a detailed representation of how different users interact with the system and its functionalities. It serves as a bridge between system requirements and system design by illustrating the flow of actions performed by users and the corresponding responses of the system [9].

The FALCONS system primarily involves two actors: the **User (Public User)** and the **Administrator**. The user represents individuals who access the system to obtain information about flood risks, weather conditions, and safety measures. These users may include residents of flood-prone areas, travelers, and tourists who need real-time updates and navigation assistance.

Users interact with the system through various functionalities, including registration and login, city selection, flood risk prediction, weather monitoring, map exploration, alert subscription, and emergency reporting

The Administrator plays a crucial role in managing the system and ensuring its proper functioning. Administrators are responsible for monitoring prediction outputs, managing user data, handling emergency reports, and issuing alerts. They also oversee system performance and ensure that all modules are operating correctly.

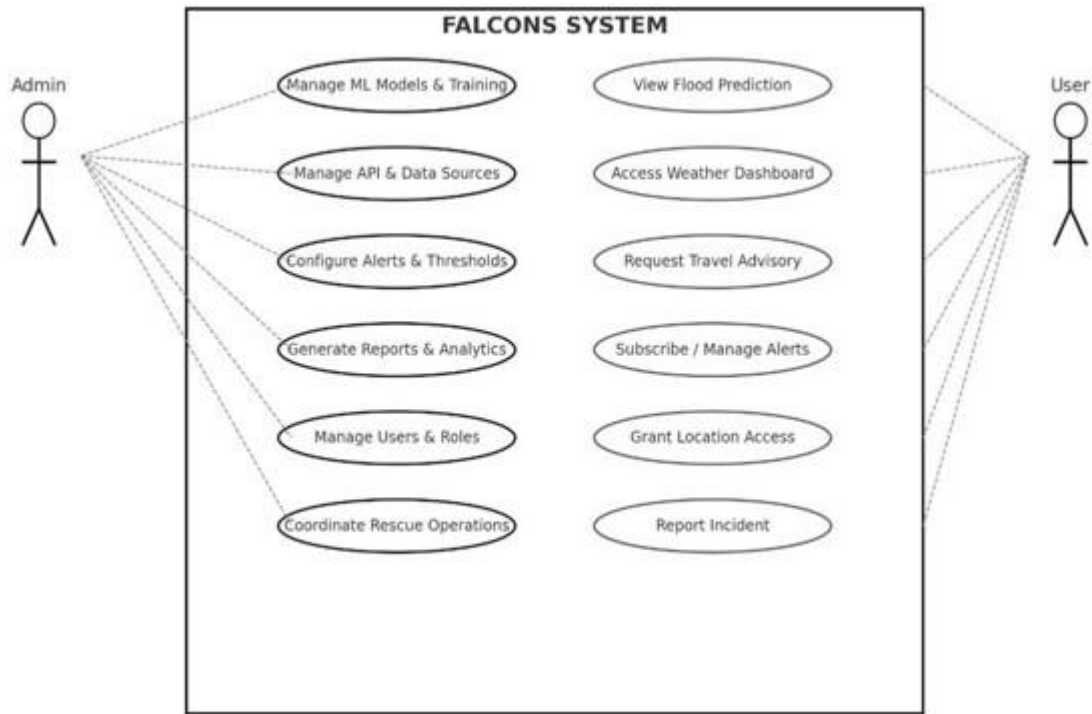


Figure 3.1-1 use case

3.1.1 Use Case Description

This case description provides a detailed explanation of system interactions and functionalities. It describes how users and administrators interact with the system under different conditions and outlines the sequence of actions performed during each interaction.

In the FALCONS system, users begin by accessing the platform and optionally registering or logging in to gain personalized features. Once logged in, users can select their city or location to receive relevant data. The system then processes environmental data and displays flood risk predictions along with weather information.

3.1.1.1 Complete Platform Interaction

Complete platform interaction provides a comprehensive overview of how users and administrators interact with the system in a real-world scenario. It defines the flow of actions, system responses, and possible variations in user behavior.

Table 3-1 complete platform interaction

Use Case ID	UC
Use Case Name	Flood Risk Prediction
Actors	Primary Actor: User Secondary Actor: Administrator, Rescue Authority
Description	This use case describes how a user interacts with the system to check flood risk based on environmental data such as rainfall, temperature, and humidity. The system processes the data and provides a risk level (Low, Moderate, High).
Trigger	The user selects a city and clicks the “Predict Flood Risk” button.
Preconditions	The user has access to a device with a camera. The AI module is active and properly integrated. The eyewear product is available and tagged for virtual try-on.
Postconditions	The system displays flood risk level along with supporting environmental data and visual indicators.
Normal Flow	Access Prediction Feature • The user accesses the FALCONS platform through the web • The user selects a city from the available list. Data Processing & Analysis • The system retrieves environmental data related to the selected city. • The prediction model processes parameters such as rainfall, temperature, and humidity. Display Output • The system displays the flood risk level using a gauge meter (Low, Moderate, High). • Supporting charts and graphs are shown to visualize environmental trends.

3.1.1.2 Normal flow

The user accesses the system interface and selects a city from the dropdown menu. After initiating the prediction process, the system retrieves relevant environmental data and processes it using the prediction model. The result is then displayed in the form of a gauge meter along with supporting charts and recommendations.

Table 3-2 normal flow

Step No.	Customer Actions	System Response
1	Registers/Login to the Platform	Validates credentials and grants access
2	Select City	Validates and accepts input
3	Requests prediction	Fetches environmental data
4	Views flood risk result	Displays risk level using gauge meter
5	Check the weather dashboard	Shows temperature, rainfall, humidity
6	Alerts	Stores alert preferences
7	Views advisory	Displays safety recommendations
8	Reports incident	Stores report and notify the admin

Step No.	Customer Actions	System Response
9	Submits a review	Saves and displays the review on the product page

3.1.1.3 Admin Flow

The administrator is responsible for managing system operations and ensuring data accuracy. This includes handling datasets, monitoring predictions, managing users, and coordinating emergency responses.

Table 3-3 Admin flow

Step No.	Admin Actions	System Response
A1	Logs into the admin panel	Verifies admin access and shows dashboard
A2	Manages datasets/Ap (add/edit/delete)	Updates environmental data sources
A3	Configures alerts	Stores alert thresholds
A4	Monitors system	Displays logs and analytics
A5	Handles rescue reports	Updates status and notifies authorities

3.2 Functional Requirements

These are the following modules that will be included in this system.

- i. User's Modules
- ii. Admin Modules

3.2.1 User's Module

- Users can register and log in securely using email and password
- Users can select their city or location to receive personalized flood predictions
- Users can view real-time flood risk levels (Low, Moderate, High)
- Users can access weather information, including rainfall, temperature, and humidity
- Users can view flood-prone areas through an interactive map interface
- Users can subscribe to alerts and receive notifications for risk updates
- Users can report flood incidents by submitting location and situation details
- Users can view safety guidelines and tourism advisory information
- Users can manage their profile information and preferences
- Users can view past alerts and prediction history

3.2.2 Admin modules

- Admin can log in securely to access the administrative dashboard
- Admin can manage user accounts (view, update, deactivate if required)
- Admin can monitor flood prediction data and system outputs
- Admin can manage environmental datasets used for prediction
- Admin can configure alert thresholds and notification settings
- Admin can view and respond to rescue reports submitted by users
- Admin can send emergency alerts and broadcast notifications
- Admin can monitor system logs and user activity

3.3 Non-functional requirement

Non-functional requirements define the quality attributes of the FALCONS system, ensuring that it performs efficiently, securely, and reliably. Some of the non-functional

3.3.1 Usability

The system is designed with a strong focus on usability, ensuring that users can easily navigate and interact with the platform. The interface is simple, intuitive, and responsive, allowing users to quickly access flood predictions and other features. Accessibility is also considered, ensuring that the platform can be used by individuals with varying levels of technical expertise. The design adapts to different screen sizes, providing consistent experience across devices.

3.3.2 Performance

Performance is critical for a real-time system like FALCONS. The platform is optimized to deliver results quickly, ensuring that users receive flood predictions without delay. Efficient data processing techniques are used to minimize response time. The system is capable of handling multiple users simultaneously, ensuring smooth operation even during peak usage.

3.3.3 Security

Security is a fundamental requirement of the system. User data is protected using secure authentication mechanisms and encryption techniques. Access control is implemented to ensure that only authorized users can access sensitive features. The system also includes validation mechanisms to prevent malicious inputs and protect against common security threats.

3.3.4 Portability

The FALCONS system is designed to be portable and compatible with multiple platforms. It can be accessed through different web browsers and devices without requiring significant modifications. The modular design of the system allows easy updates and integration with new technologies, ensuring long-term sustainability.

3.3.5 Reliability

Reliability ensures that the system operates consistently under different conditions. The FALCONS system uses structured databases along with efficient data handling mechanisms to maintain accuracy and consistency.

The system is designed to minimize failures and ensure continuous availability, especially during critical situations where timely information is essential.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The design and architecture of FALCONS (Flood Alert, Climate Observation & Navigation System for Safety) are centered around developing a modular, scalable, and intelligent web-based platform specialized for flood risk prediction and disaster response. The frontend ensures a responsive and intuitive user experience for viewing flood risk levels, accessing real-time weather data, managing safety alerts, and navigating rescue resources. The backend, built using FastAPI (Python), is responsible for secure data handling, real-time ML inference, and well-structured module integration, including the flood prediction engine, alerts management, geo-visualization, and rescue resource tracking. This cohesive architecture supports smooth operations, intelligent risk scoring, and personalized safety advisories, enhancing both public awareness and authority-level disaster response efficiency [3].

4.1 System Architecture

Diagram 4.1 illustrates how FALCONS serves users through a web-based platform that integrates real-time flood prediction with API-driven weather data and geospatial mapping. At its core, the system supports essential features such as user registration/login, flood risk visualization, real-time weather dashboards, tourism safety advisories, alert subscription management, and rescue resource tracking. For authorities and administrators, it includes modules for resource management, road status updates, alert log monitoring, and direct alert broadcasting [4].

Users interact with the platform through intuitive web interfaces, accessing features such as flood risk gauges and interactive flood maps. The backend, powered by FastAPI (Python), processes these interactions and communicates with external APIs (OpenWeatherMap, NASA IMERG), the ML prediction engine, and the Supabase (PostgreSQL) database for real-time risk calculation and secure data management. All transactions and activities are routed through a central server that ensures real-time performance, scalability, and data security, allowing seamless user access and efficient authority operations.

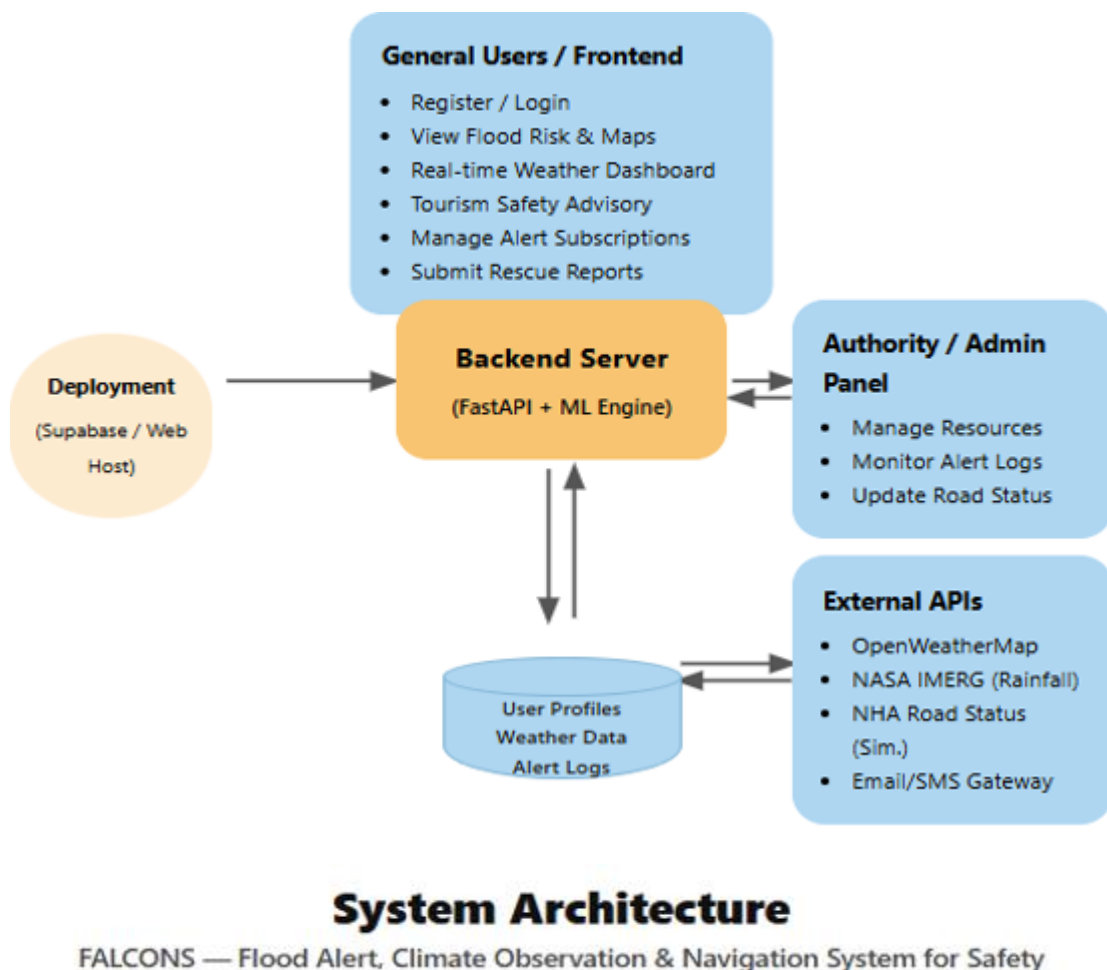


Figure 4.1-1 System architecture

The system architecture of FALCONS diagram 4.1-1 is designed to provide a seamless and intelligent user experience by integrating ML-powered flood prediction with real-time weather data and interactive geospatial visualization. The architecture follows a three-tiered structure: the Frontend Layer, the Backend Layer, and the Database Layer.

The Frontend Layer acts as the primary user interface, allowing visitors and registered users to interact with the platform. It includes modules such as the Flood Risk Gauge, real-time weather dashboard, interactive flood map (Leaflet.js), tourism safety advisory, alert subscription management, and rescue resource console all designed to be responsive and user-friendly across

The Backend Layer handles the core application logic and services. It processes user input, runs the ML flood prediction model, fetches and caches external API data, manages user authentication via JWT, and coordinates real-time alert triggering and notification delivery. This layer ensures secure and efficient communication between the frontend and the database, enforcing all business rules and application workflows [6].

The Database Layer, powered by Supabase (PostgreSQL), securely manages and stores data such as user profiles, weather records, flood predictions, alert logs, and rescue resource locations. Each component in the backend interacts with structured database entities to maintain data integrity, optimize performance, and provide real-time updates across the platform [8].

4.2 Data Representation

This architecture diagram 4.2-1 ensures data integrity, consistency, and high performance across all interconnected modules, including flood prediction, weather monitoring, alert management, and rescue resource deployment. Given the interactive and feature-rich nature of the platform, which includes ML-driven flood risk scoring, user registration, real-time weather display, alert subscriptions, tourism safety advisories, and rescue resource management, the system relies on predefined database schemas to manage data efficiently across various components. Each core entity, such as users, weather data, flood predictions, alert logs, subscriptions, and rescue resources, is stored in well-structured tables with clear primary and foreign key relationships. This structure allows the system to accurately map user interactions with risk data and services, maintain consistent data flow, and ensure reliability during real-time operations [10].

The platform utilizes Structured Query Language (SQL) via Supabase (PostgreSQL) to execute complex queries, data joins, filters, and aggregations, all of which are essential for dynamic risk calculation, personalized alert triggering, and detailed operational analytics. This robust data layer is key to delivering a fast, secure, and intelligent safety platform. While FALCONS primarily relies on a structured relational database model, it also incorporates semi-structured formats like JSON in specific areas. These include storing ML model metadata, rescue resource geo-coordinates, real-time API response caches, and user-submitted incident report details [2].

This hybrid approach balances the reliability and efficiency of SQL with the flexibility and

adaptability of lightweight data formats. JSON is particularly useful in managing variable data structures such as NASA IMERG rainfall payloads or ML model performance metrics, which can vary in structure and size. By combining structured and semi-structured data models, the platform ensures robust performance, enhanced scalability, and the ability to adapt quickly to evolving features, all while maintaining a seamless and responsive user experience across the entire system.

FALCONS – Data Representation

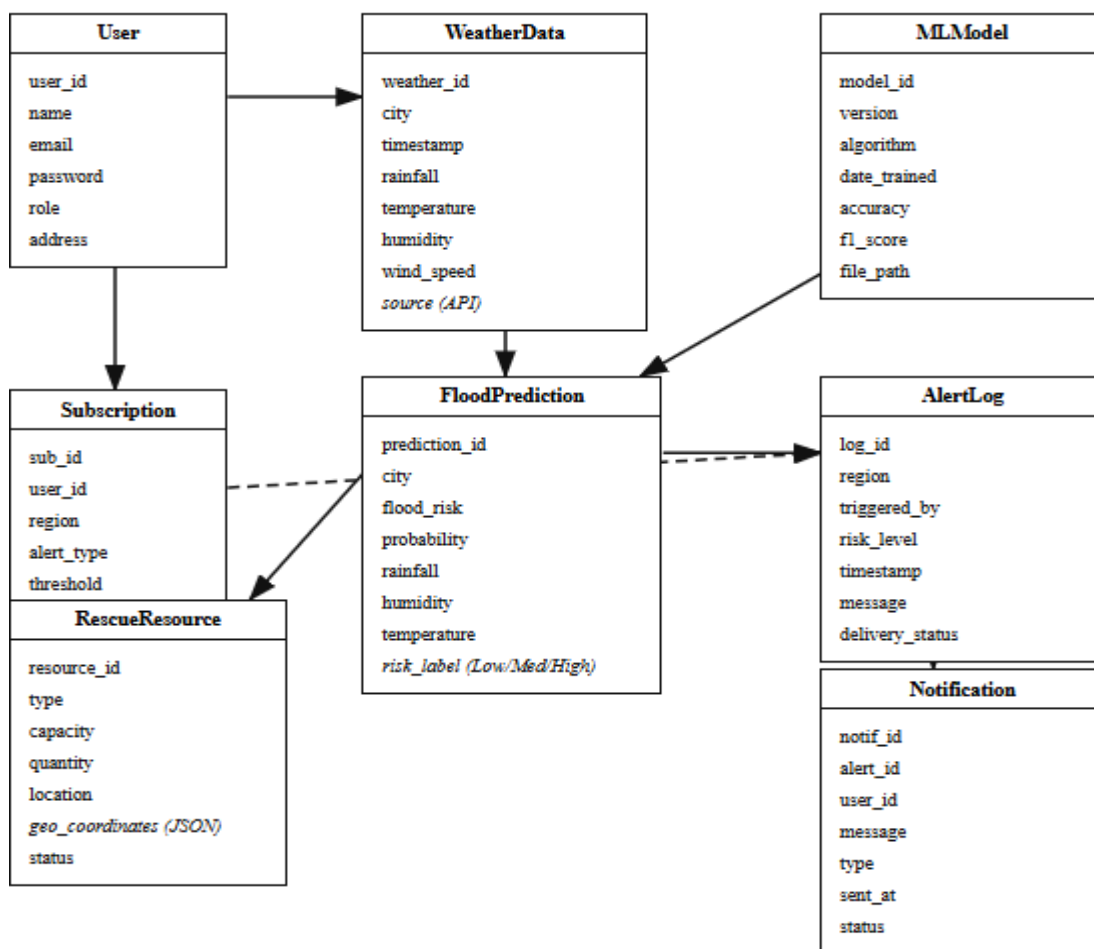


Figure 4.2-1 Data representation

4.3 Process Flow/Representation

The FALCONS platform process flow begins with user registration and authentication, providing secure access to features such as flood risk visualization, real-time weather data, tourism safety advisories, and alert subscriptions. Authorities access a dedicated console to manage resources, update road status, and monitor alert logs. The system ensures smooth data flow between the frontend and backend, supporting real-time ML inference, automated alert triggering, and seamless management of user interactions and disaster response operations.

4.3.1 Activity Diagrams

Activity diagrams play a vital role in modeling the dynamic behavior of a system, offering a comprehensive visual representation of how various actions flow and interact. These diagrams depict sequential, branching, and parallel workflows, using constructs such as decisions, forks, joins, and transitions to represent concurrent and conditional activities effectively.

They are especially useful for illustrating how different components within a system coordinate and evolve, making complex processes easier to understand. This clarity enhances system analysis, design, and stakeholder communication. Moreover, activity diagrams help identify inefficiencies, bottlenecks, or redundancies in workflows. By visualizing all possible action flows, they enable better process optimization and facilitate forecasting the impact of changes on system performance.

4.3.2 Admin Activity Diagram

The Admin / Authority Activity Diagram illustrates the essential tasks performed by the administrator within the FALCONS platform. It begins with the authority logging into the system and accessing the Authority Console. From there, the admin can:

- **Manage Resources:** Add, edit, or delete rescue resource locations.
- **Monitor Alert Logs:** Review all system-generated warnings, inspect risk levels by region and timestamp, verify delivery status, and export logs for compliance and auditing.
- **Broadcast Alerts:** Manually trigger UNSAFE alerts to subscribed users for specific regions when risk levels demand immediate action beyond automated thresholds.
- **Manage Rescue Reports:** Review incident reports submitted by general users and update report status.

Diagram 4.3-1 illustrates a streamlined flow from user authentication to specific administrative operations, reinforcing system oversight and ensuring a high-quality, timely disaster response experience. By mapping out clear pathways for administrative actions, the diagram highlights how the system minimizes operational friction during critical moments. This structured approach ensures that administrators can quickly manage resources, monitor alert logs, and broadcast emergency warnings without facing procedural delays. Ultimately, this seamless workflow bridges the gap between backend data management and real-time field operations, maximizing overall system efficiency when every second counts.

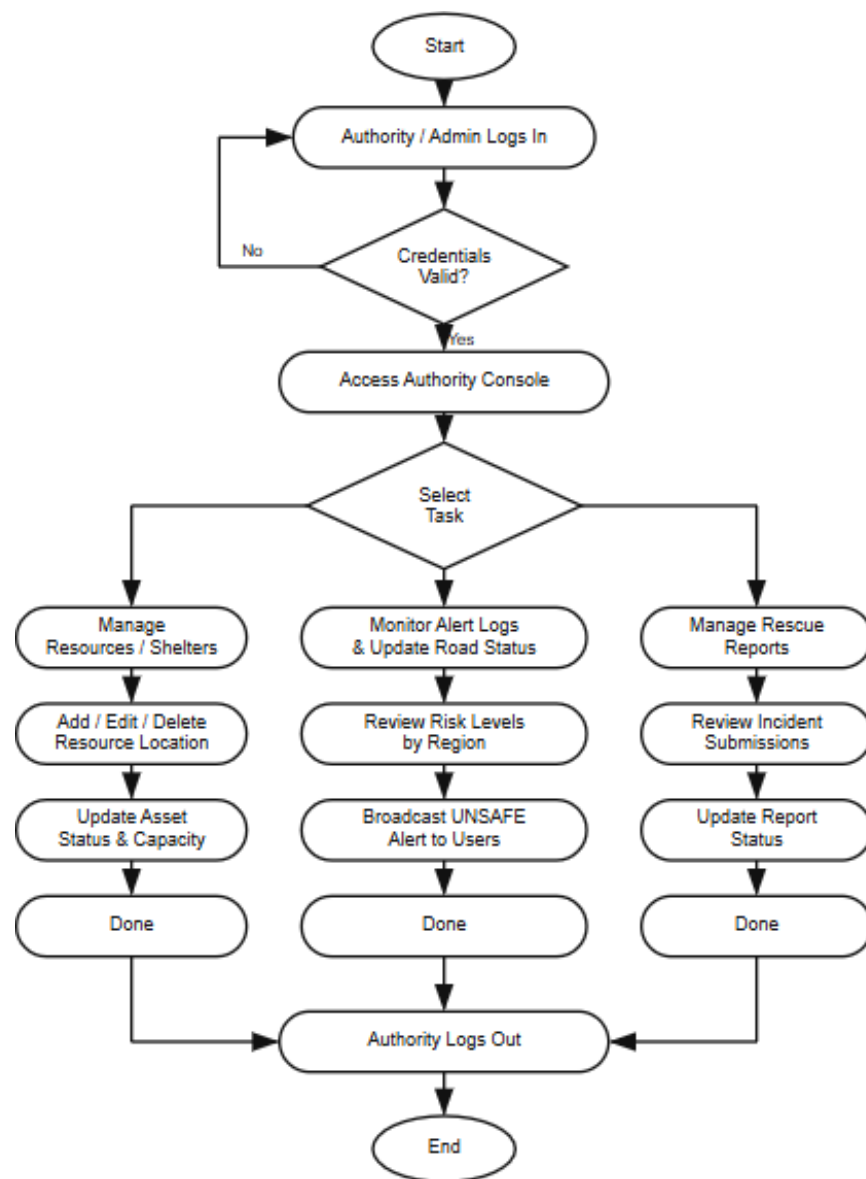


Figure 4.3-1 Admin activity

4.4 Process Flow

Diagram 4.4-1 illustrates visual representation that maps out the sequential steps, decisions, and interactions within a system or workflow. It highlights how inputs move through various stages of processing, showing the relationships and data exchange between users, system components, and external services. The purpose of a PFD is to provide a clear, high-level overview of the entire operation, enabling stakeholders to understand how the system functions as a whole and identify any potential inefficiencies or bottlenecks.

Here is the Process Flow Diagram (PFD) for the FALCONS project, which clearly outlines the detailed sequence of user interactions and system operations that deliver real-time flood safety intelligence. This diagram illustrates how the user initiates actions such as logging in or registering, and how this input flows through the system to present flood risk scores, weather data, and safety advisories. It further depicts how the ML Prediction Engine (M1) receives fresh API data, computes a probabilistic risk score (0–100%), and how the Notification Service checks subscriptions against personalized thresholds to trigger automated Email/SMS alerts. Additionally, the flow shows the interaction between the user interface, FastAPI backend, external APIs, the database, and the Authority Console highlighting how these components seamlessly coordinate to provide a smooth and life-saving user experience. The diagram provides a comprehensive overview of the end-to-end process, from initial user engagement to real-time alert delivery, ensuring clarity on how each module contributes to the platform's functionality. The complete process flow diagram is as follows:

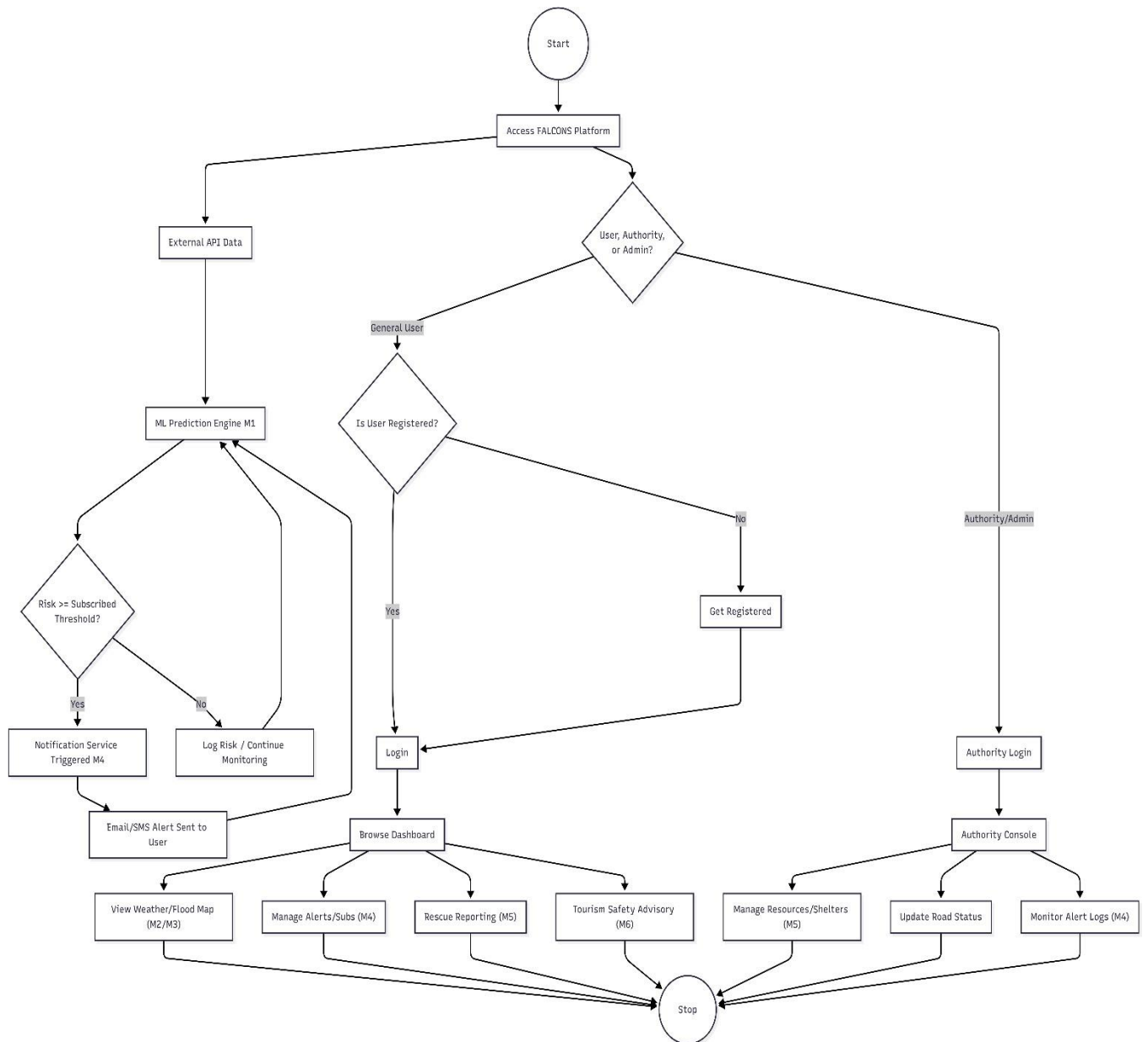


Figure 4.4-1 Process flow

4.5 Design Models

The design model of FALCONS provides a structured blueprint outlining how the system's core components, such as user management, ML-powered flood prediction, weather data integration, alert management, geo-visualization, and the authority console, interact to deliver a seamless and intelligent disaster safety platform. It emphasizes a user-centric interface that allows easy navigation and real-time risk visualization through an interactive flood map. The model details the data flow between the frontend and backend, ensuring smooth integration of FastAPI for backend processing, Supabase (PostgreSQL) for data storage, and ensemble ML algorithms for accurate flood probability calculation. Security measures for JWT-based authentication and scalability considerations are embedded to support future growth, making the platform reliable, efficient, and life-saving for both general users and disaster response authorities [1].

4.5.1 Sequence Diagram

The Sequence Diagram 4.5.1-1 of the FALCONS project comprehensively illustrates the detailed interaction flow between the user and the system's key components, capturing the essential functionalities of the platform. It outlines each step the user takes, starting from accessing the website, through authentication processes such as login or registration, to engaging with the flood risk dashboard and alert subscription features. The diagram further depicts the automated system flow where the External API sends fresh rainfall data to the FastAPI backend, the Flood Prediction Engine (M1) loads the ML model and calculates a new probabilistic risk score for a specific region, and the Notification Service queries the database (Supabase) to identify subscribed users whose personalized risk threshold is met. Crucially, it shows the coordinated communication between the user interface, backend services including the authentication module (JWT), the ML prediction and notification services, and the database for storing user profiles, alert logs, and resource data. By mapping these interactions in sequence, the diagram provides a clear blueprint of the system's dynamic behavior, ensuring that all processes are logically ordered and optimized to meet user safety needs effectively and efficiently. The sequence diagram of our project is as follows:

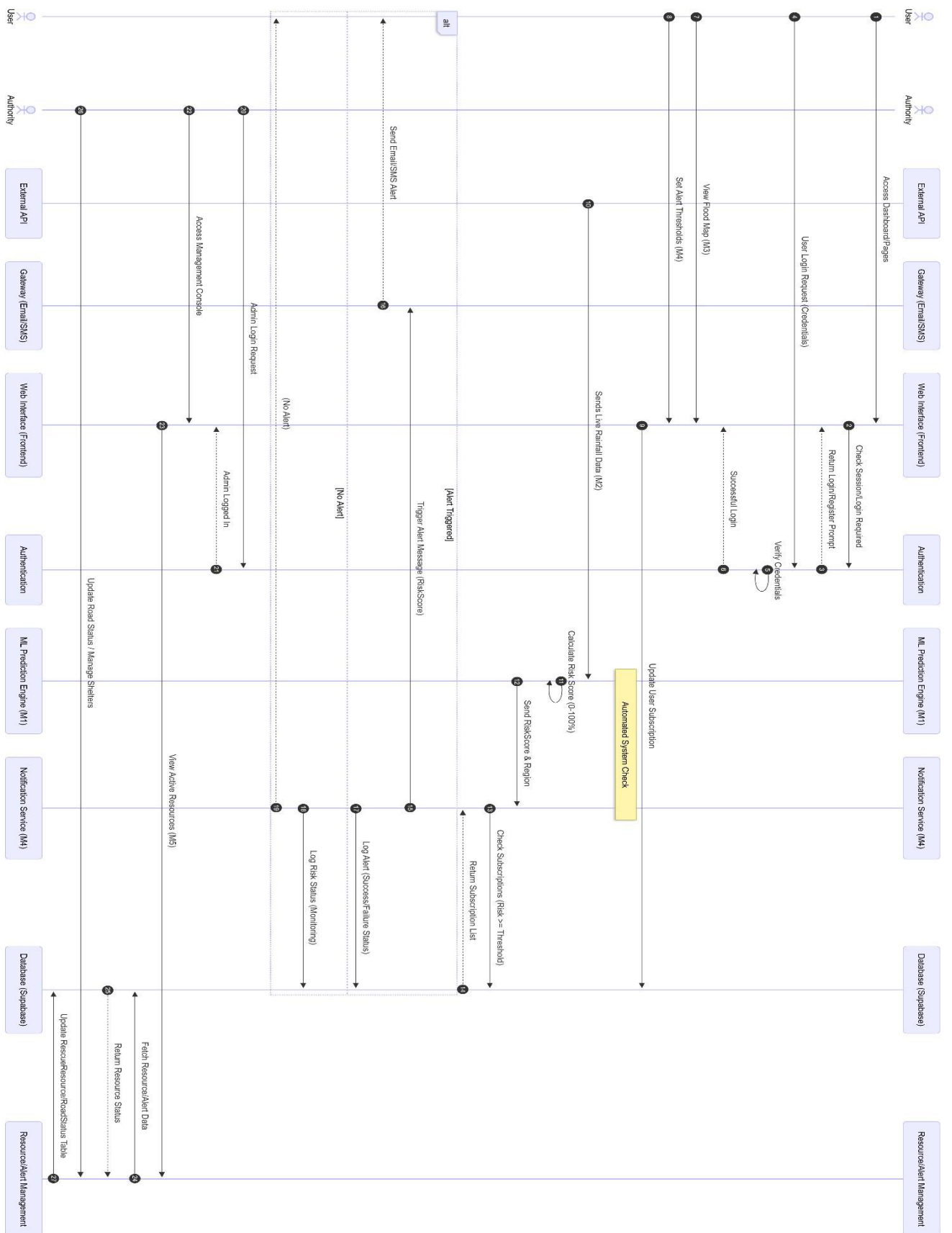


Figure 4.5-1 Sequence Diagram

4.5.2 Class Diagram

A Class Diagram is a type of UML (Unified Modeling Language) diagram used to represent the static structure of a system by showing its classes, attributes, methods (functions), and the relationships between objects. It is a foundational blueprint in object-oriented design and is especially useful for understanding how different parts of a system interact at a structural level.

Class diagrams provide a clear map of:

- What data each class holds (attributes),
- What actions each class performs (methods),
- How classes are connected (associations, inheritance, dependencies),
- And how data flows or is grouped across the system (aggregation, composition).

The Class Diagram for the FALCONS project provides a detailed visualization of the system's static architecture by illustrating its core classes, attributes, methods, and the relationships among them. It encapsulates the structure of key entities such as User, Authentication, Weather Service, Admin, Alert System, Flood Map, Subscription, Flood Prediction, Weather Data, Flood Data, Rescue Report, Tourism Advisory, Notification, Emergency Response Team, and City Data. Each class defines specific attributes (e.g., User has user ID, name, email, password; Flood Prediction has city, flood Risk, humidity, rainfall) and methods (e.g., register(), login(), computeRisk(), broadcastAlert(), sendNotification()) relevant to its functionality [1].

The diagram showcases associations (e.g., a User can manage multiple Subscriptions), dependencies (e.g., Flood Prediction depends on WeatherData), and aggregations (e.g., Admin aggregates Alert System and FloodMap management). Additionally, it represents one-to-many and many-to-many relationships where applicable, such as between users and their alert subscriptions or between flood predictions and city data [2].

This class diagram serves as a blueprint for the database schema and object-oriented programming logic. It ensures consistency across development, facilitates code maintenance, and provides a clear understanding of the system's components and their interconnections [5].

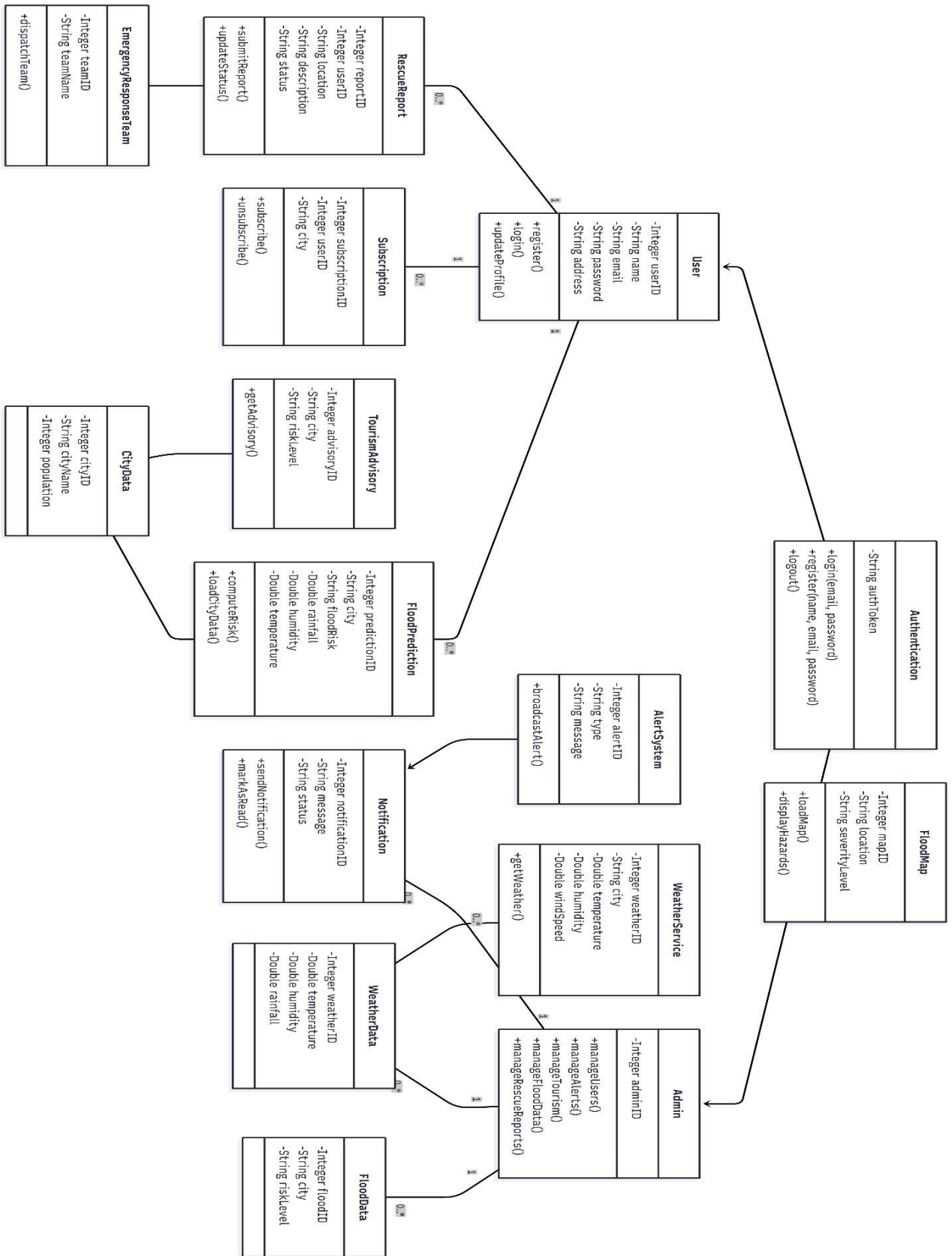


Figure 4.5-2 Class diagram

4.6 Data Design

As we have discussed all the aspects of FALCONS, now we will discuss the data design of our project. It describes the whole data structure of our application to ensure an efficient and secure system.

- **User Information** User Information will store details about users, including their names, contact information, passwords (securely encrypted), and addresses. It will also include user roles (Admin, Authority, or General User) to manage permissions effectively.
- **Subscription Information** Subscription Information will maintain details about the user's active alert configuration, recording the user's selected geographical region, the alert type they prefer (Flood, Snow, or Rain), their customized risk threshold, and their preferred contact method (Email/SMS).
- **Admin / Authority Information** Admin / Authority Information will maintain details about administrators managing the application, including credentials and activity logs to monitor system interactions and track resource management actions.
- **Alert-Log Information** Alert-Log Information will maintain records of all system-generated warnings, tracking the alert's risk level, timestamp, delivery status, triggering source (ML Model or Admin), and the region that triggered the alert.
- **Rescue Resource Information** Rescue Resource Information will store data about critical assets available for deployment, such as the asset's type (Shelter, Medical Camp, Rescue Vehicle), capacity, stock quantity, current status, and exact geo-spatial coordinates.
- **Weather Data Information** Weather Data Information will store features fetched from external APIs (OpenWeatherMap, NASA IMERG), including live rainfall, temperature, humidity, and wind speed — serving as essential inputs for the ML model's inference calculations.
- **Flood Prediction Information** Flood Prediction Information will store the output of each ML inference cycle, including the city, computed flood probability (0–100%), risk label (Low/Moderate/High), and the input feature values used.
- **ML Model Information** ML Model Information will maintain records of the predictive models used, including version ID, algorithm type, training date, accuracy, F1 score, and file path.
- **Database Security** Database Security measures, such as JWT-based user authentication and secure data storage via Supabase (PostgreSQL), will be implemented to protect all sensitive user and operational information.

4.7 Data Dictionary

The Data Dictionary for the FALCONS (Flood Alert, Climate Observation & Navigation System for Safety) project is a list of the names and attributes used within the system. These elements define the key components that will be implemented in the project. The dictionary includes details such as data types, formats, and relationships between different entities in the system.

- **User:** A person who visits the website to access flood risk information, manage safety alerts, or monitor disaster resources. Attributes: Name, Contact Information, Role, Location, Login Credentials (username, hashed password).
- **Alert Log:** A record of all system-generated warnings, used for compliance, auditing, and displaying historical alert history. Attributes: Log ID, Region Name, Triggered By (ML Model or Admin), Risk Level (High, Medium, Low), Timestamp, Message, Delivery Status.
- **Rescue Resource:** Information about assets available or deployed in flood-affected areas for aid and relief operations. Attributes: Resource ID, Type, Capacity / Quantity, Status (Available / Deployed / Full).
- **Weather Data:** Weather Data: Data fetched from external, crucial for feeding the flood prediction engine and displaying current conditions on the dashboard. Attributes: Data ID, City, Timestamp, Rainfall (mm), Temperature (°C), Humidity (%), Wind Speed (km/h), Snowfall Volume (mm).
- **Flood Prediction:** The computed output of each ML inference run for a specific region. Attributes: Prediction ID, City, Flood Risk, Flood Probability (0–100%), Rainfall, Humidity, Temperature, Timestamp.
- **ML Model:** Information about the machine learning model used for real-time flood probability calculation. Attributes: Version ID, Algorithm (e.g., Random Forest, Ensemble), Date Trained, Accuracy (%), F1 Score, File Path.
- **User Report (Incident):** User Report (Incident): Details submitted by a general user regarding a live flood or emergency requiring aid. Attributes: Report ID, User ID, Timestamp, Location (Geo-coordinates), Description, Severity Level, Status
- **Admin Activity Log:** Admin Activity Log: A collection of records tracking actions performed by Authority / Admin roles within the platform. Attributes: Log ID, Admin ID, Timestamp, Action Type Details.

Chapter 5

IMPLEMENTATION

5. Implementation

The implementation phase of the **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)** project focuses on transforming the system design into a fully functional, intelligent, and scalable web-based platform for flood prediction, monitoring, and emergency response. This phase ensures that all defined modules, including flood prediction, weather monitoring, alert systems, tourism safety advisory, and rescue reporting, are developed, integrated, and deployed effectively [6].

The system is developed using a modular and incremental approach, allowing each component to be independently implemented and tested before integration. The backend is responsible for handling real-time data processing, prediction algorithms, and system logic, while the frontend provides an intuitive interface for users to interact with maps, alerts, and reports [4].

FALCONS integrates machine learning concepts for flood prediction, real-time weather data processing, and user-generated incident reporting. The system ensures efficient communication between users, administrators, and rescue authorities. Key functionalities include user authentication, flood risk visualization, alert broadcasting, location-based services, and reporting mechanisms [1].

5.1 Flowchart

5.1.1 User Authentication

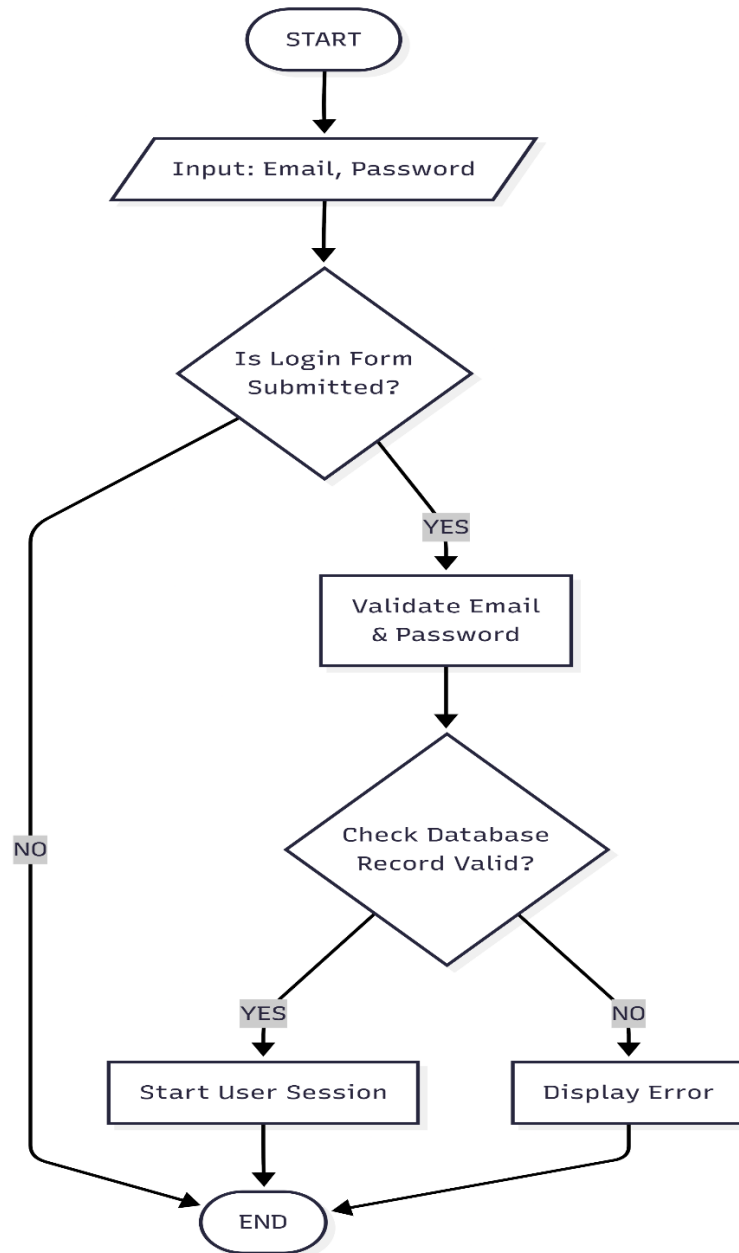


Figure 5.1.1 User Authentication

5.1.2 Flood Prediction Flowchart

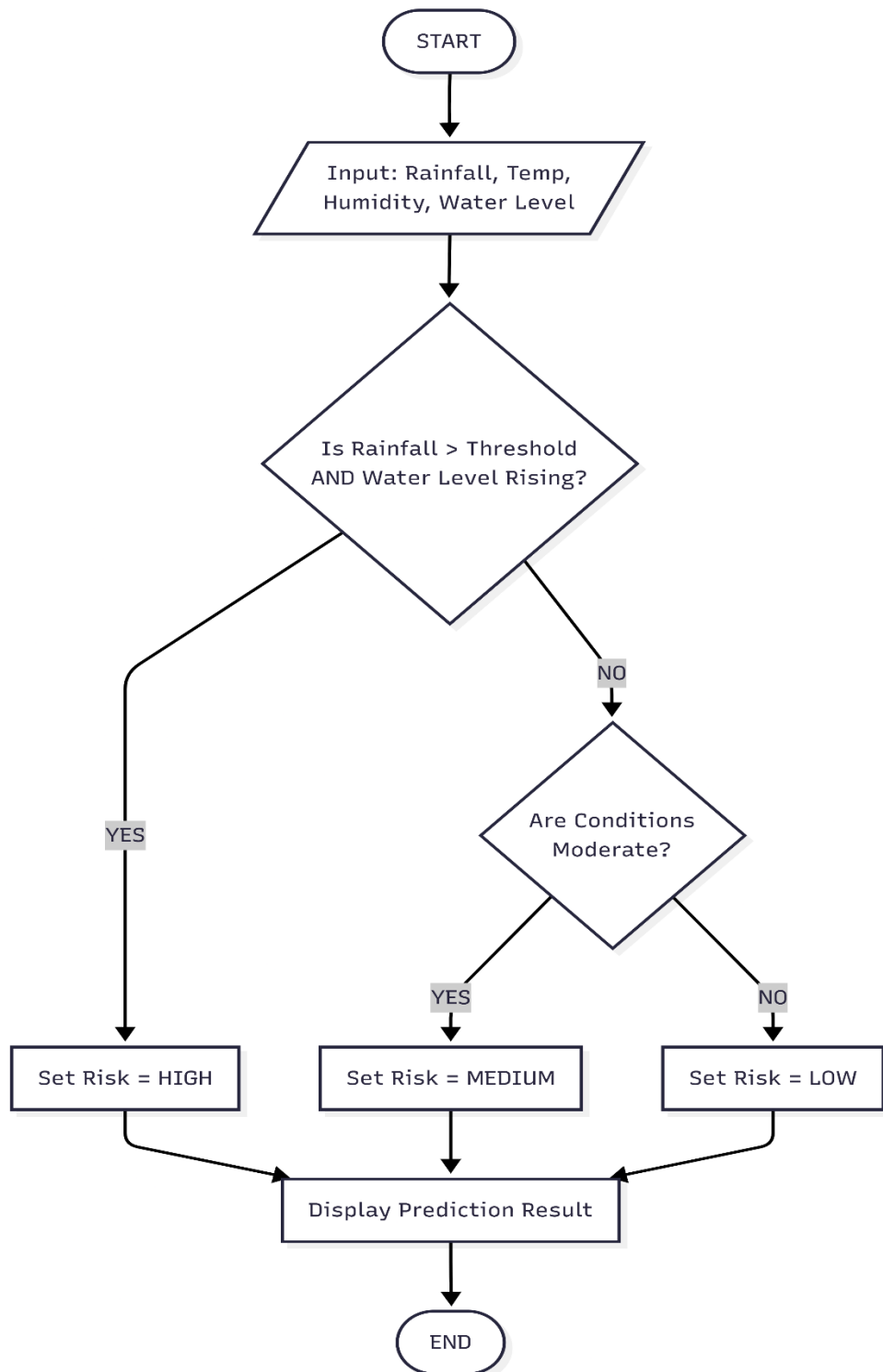


Figure 5.1.2 Flood prediction

5.1.3 Alert Generation System

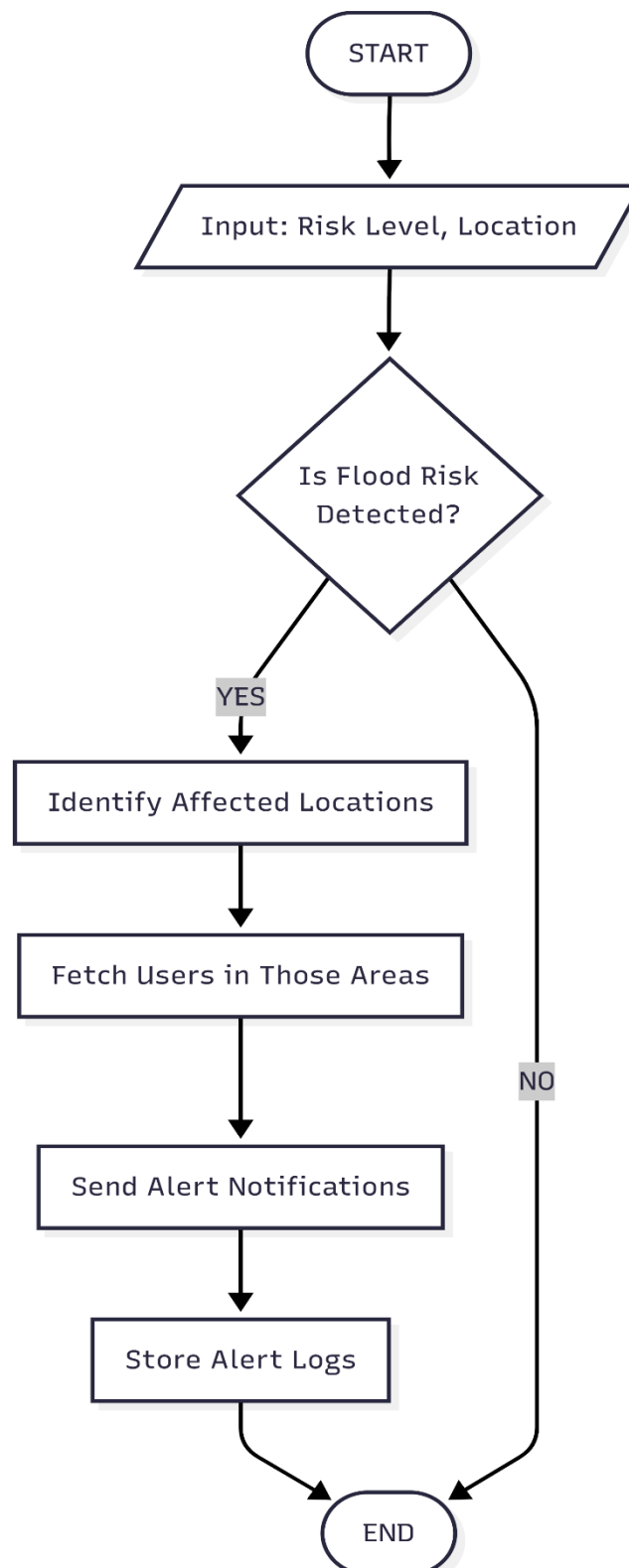


Figure 5.1.3 Alert Generation System

5.1.4 Admin Management & User Management

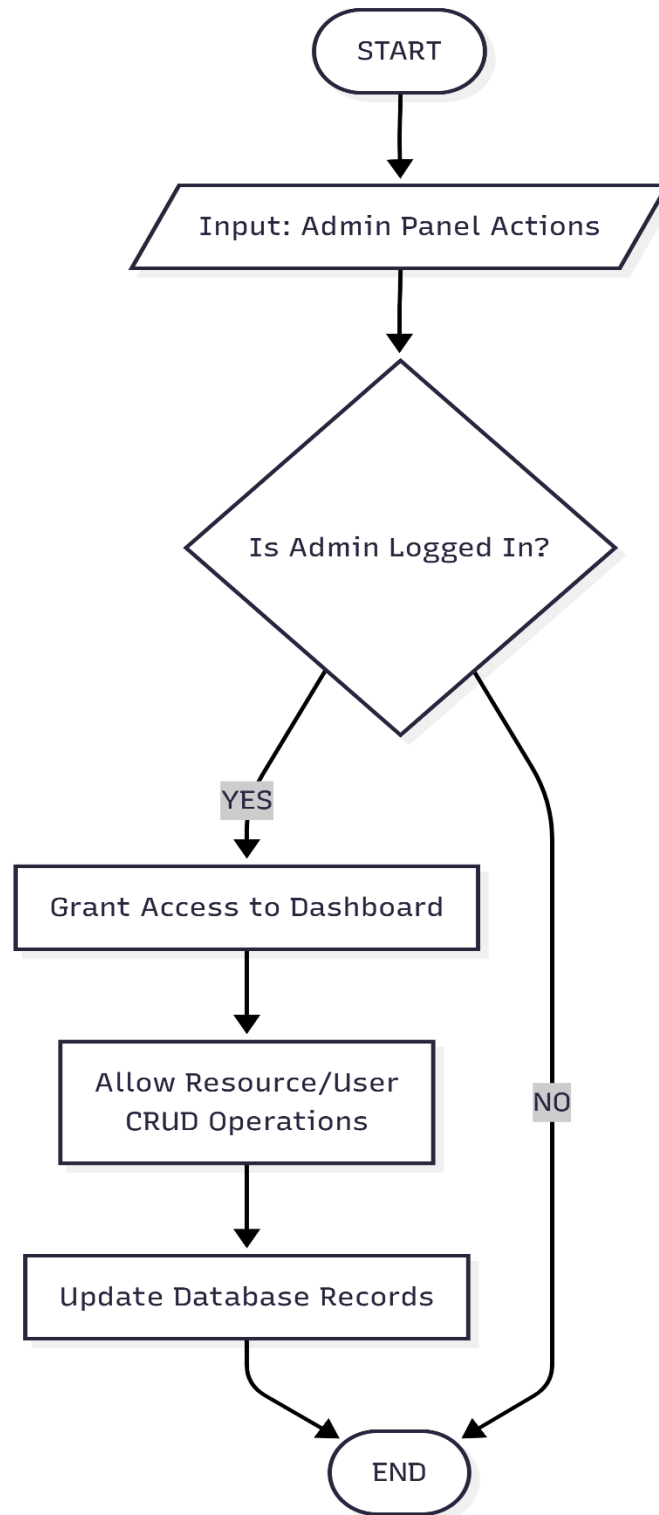


Figure 5.1.4 Admin Management

5.1.5 Rescue Reporting System

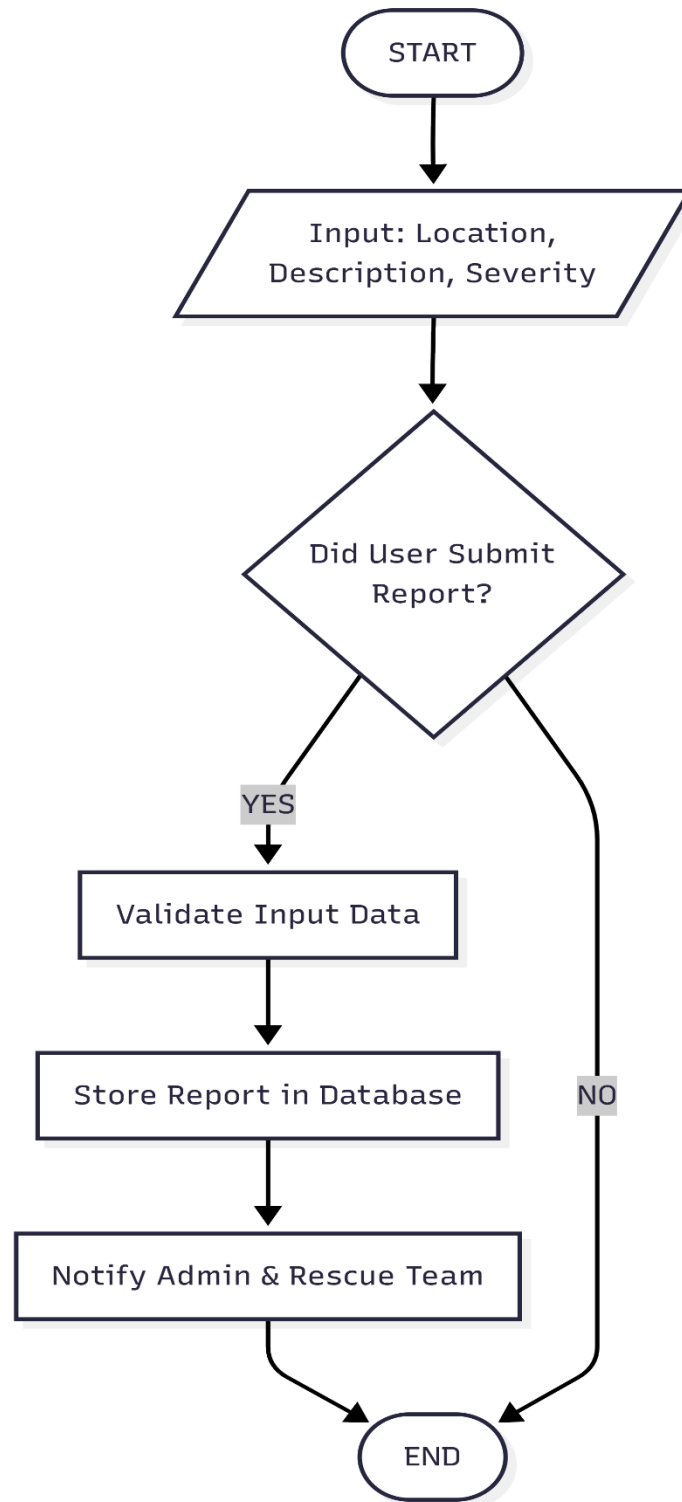


Figure 5.1.5 Rescue Reporting System

5.2 External APIs

The FALCONS platform is designed using a simplified and efficient architecture that relies primarily on **Supabase** as the backend service. Instead of traditional backend frameworks such as Node.js or Laravel, Supabase provides an integrated solution that includes database management, authentication, APIs, and real-time capabilities [10].

Supabase is built on PostgreSQL and offers a cloud-based backend that eliminates the need for managing a separate server. This approach simplifies development, reduces system complexity, and improves scalability. All core functionalities, such as user authentication, data storage, flood reports, alert logs, and system records, are handled through Supabase services [9].

The platform uses frontend technologies such as HTML, CSS, and JavaScript to interact directly with Supabase APIs. Real-time features, such as alert notifications and rescue reporting updates, are supported through Supabase's built-in real-time subscriptions. Security is managed through Supabase authentication and role-based access control policies [7].

Table 5-1 External API

Component	Technology / Tool	Purpose / Description
Backend & Database	Supabase (PostgreSQL)	Stores user data, flood data, alerts, and reports
AI-Based Try-On System	Supabase Auth	Handles login, registration, and session management.
Frontend UI	HTML, CSS, JavaScript	User interface and interaction
Real-Time Features	Supabase Realtime	Live alerts and updates
Security Database	Supabase Policies (RLS)	Ensures secure data access control
Authentication	Supabase Authentication	Manages user registration, login, and session handling without third-party OAuth integrations.
Hosting	(Your hosting e.g., Verel / Netlify)	Deploys frontend application

5.3 User Interface

The user interface of the FALCONS system is designed to provide a clear, responsive, and user-friendly experience, especially during emergency situations. Each screen is structured to ensure that users can quickly access critical information such as flood predictions, alerts, and reporting tools. The following figures illustrate the major system interfaces.

5.3.1 Home Screen

Figure 5.3.1 illustrates the Home Screen of the FALCONS platform, which serves as the main entry point for users. This screen provides access to key features such as flood prediction, weather monitoring, alert notifications, tourism safety information, and rescue reporting. The interface is designed with a structured navigation system that allows users to move easily between different modules. Important alerts and system updates are displayed prominently to ensure users are aware of critical conditions. The layout is simple, responsive, and user-friendly, making it accessible for individuals with varying levels of technical expertise. Additionally, the screen includes login and registration options to enable personalized access to system functionalities.

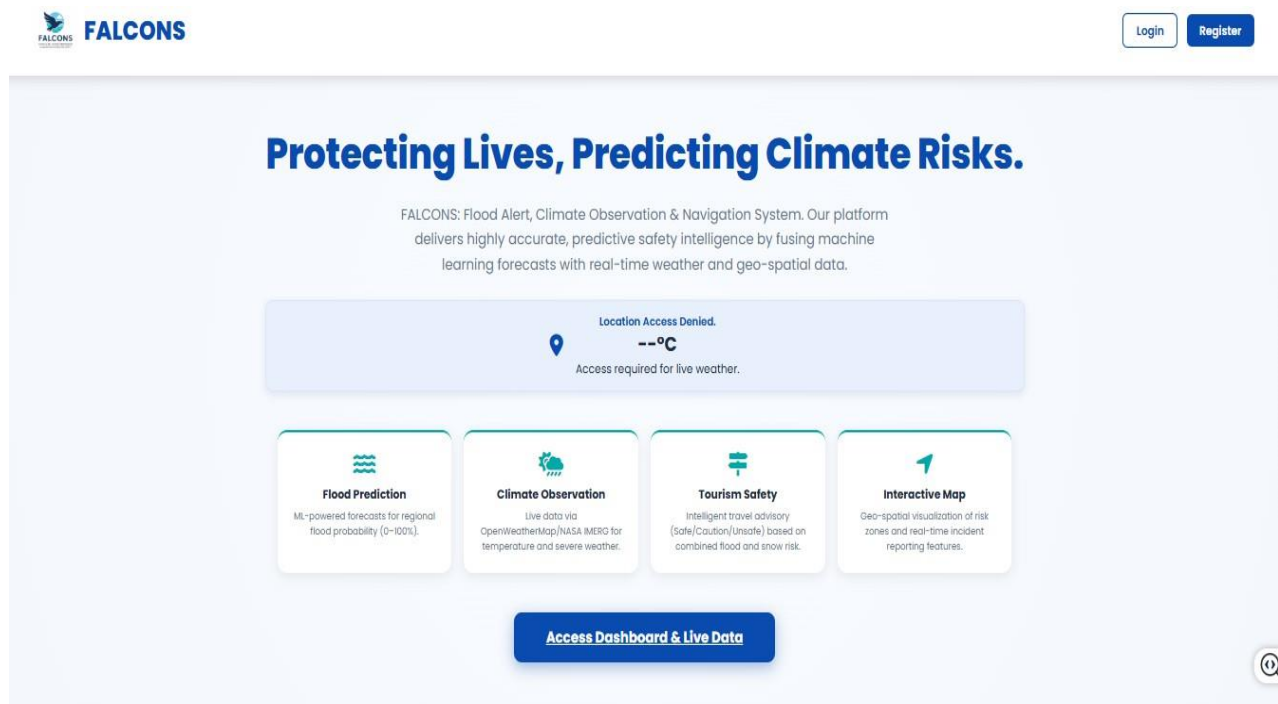


Figure 5.3-1 Home Page

5.3.2 Dashboard Screen

Figure 5.3.2 illustrates the Dashboard Screen of the FALCONS platform, which provides a centralized overview of system information. This screen displays key data such as current weather conditions, flood risk levels, and recent alerts in a structured manner. It enables users to quickly understand the overall situation without navigating multiple sections. Visual elements such as indicators and charts are used to improve clarity and decision-making. The dashboard acts as a control center for both users and administrators. Its organized layout ensures efficient access to all major system functionalities.

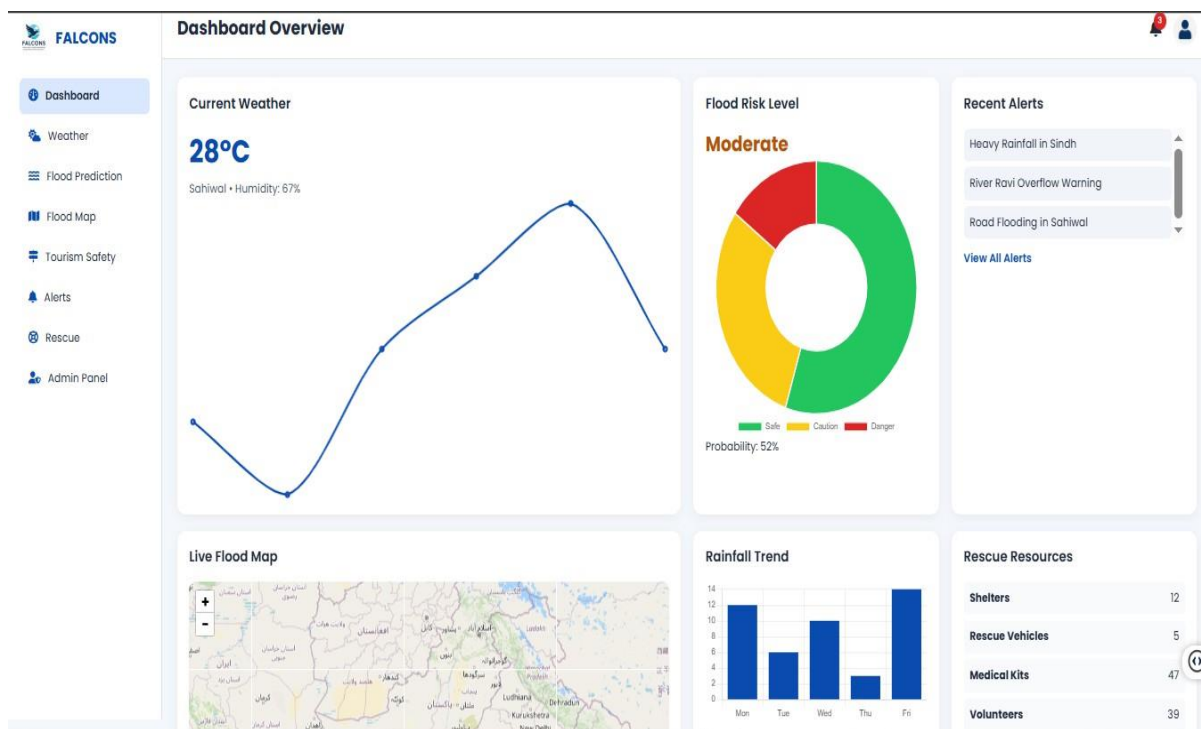


Figure 5.3-2 Dashboard Page

5.3.3 Weather overview screen

Figure 5.3.3 illustrates the Weather Overview Screen, which presents real-time environmental data including rainfall, temperature, humidity, and wind conditions. This information is essential for analyzing patterns that may lead to flooding. The data is displayed using graphical representations to enhance user understanding. The system continuously updates the values to maintain accuracy and reliability. Users can observe trends and variations over time through charts. This screen plays a supporting role in improving the accuracy of flood prediction.

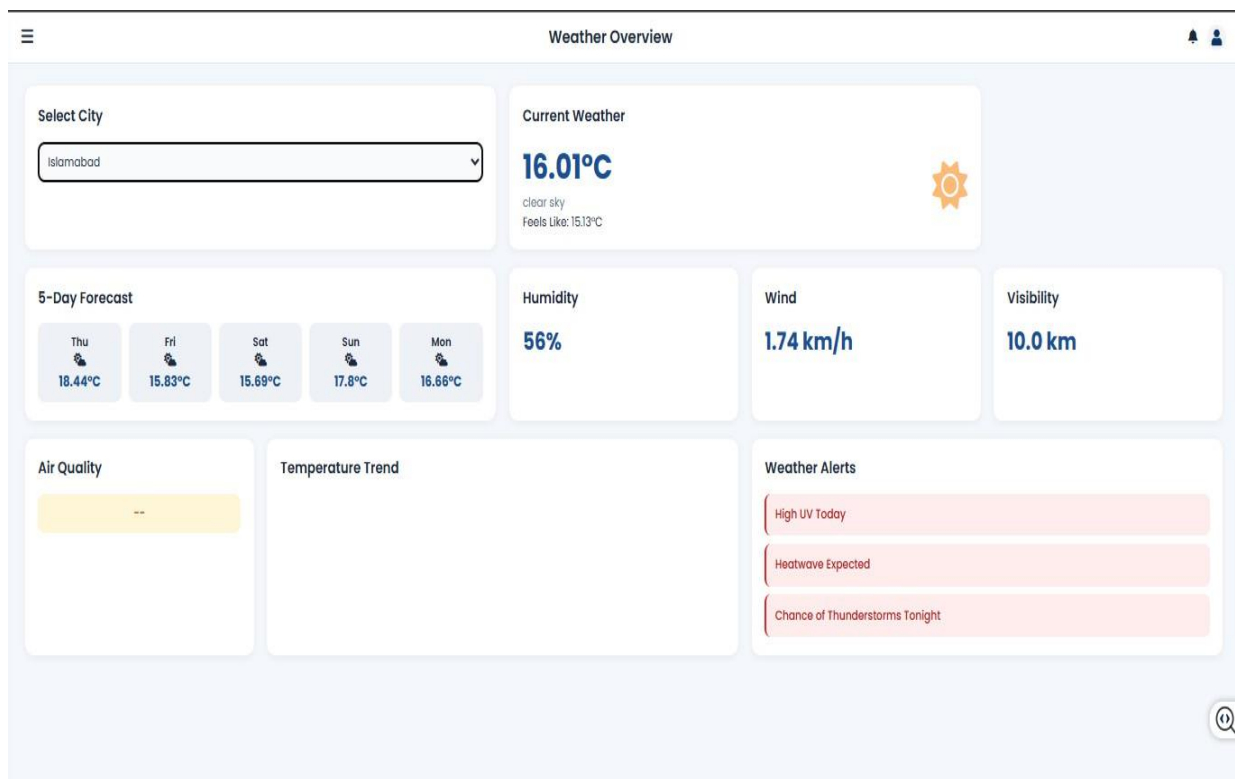


Figure 5.3-3 Weather page

5.3.4 Flood prediction

Figure 5.3.4 illustrates the Flood Prediction Screen, which is one of the core components of the FALCONS platform. It provides users with flood risk analysis based on processed environmental data. The system categorizes risk levels into low, moderate, and high using clear visual indicators. Users can select specific cities to view localized predictions. The interface uses color-based signals to highlight critical conditions effectively. This feature helps users take preventive measures and improve disaster preparedness.

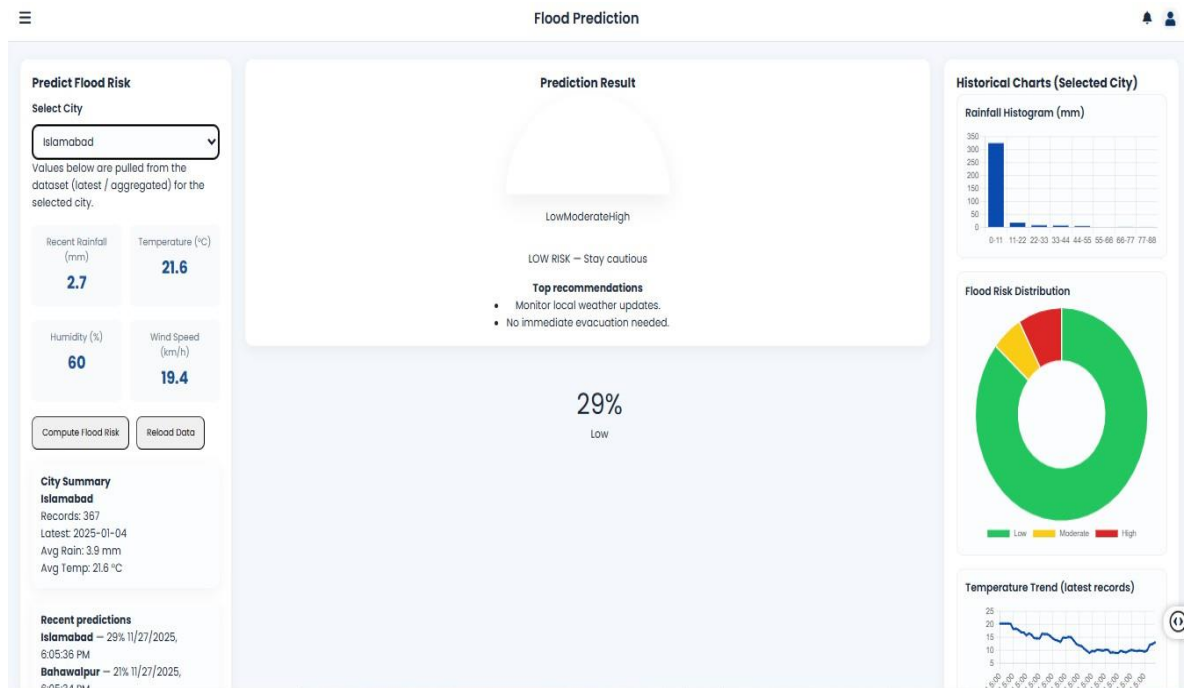


Figure 5.3-4 Flood Prediction

5.3.5 Tourism Safety Screen

Figure 5.3.5 illustrates the Tourism Safety Screen, which provides safety recommendations for travelers based on current weather and flood conditions. This screen identifies safe and high-risk areas to help users make informed decisions. The system updates information dynamically to reflect real-time environmental changes. Users can review advisories before planning travel activities. The interface ensures that safety-related information is clearly visible and easy to understand. This module contributes to reducing risks associated with travel during adverse conditions.

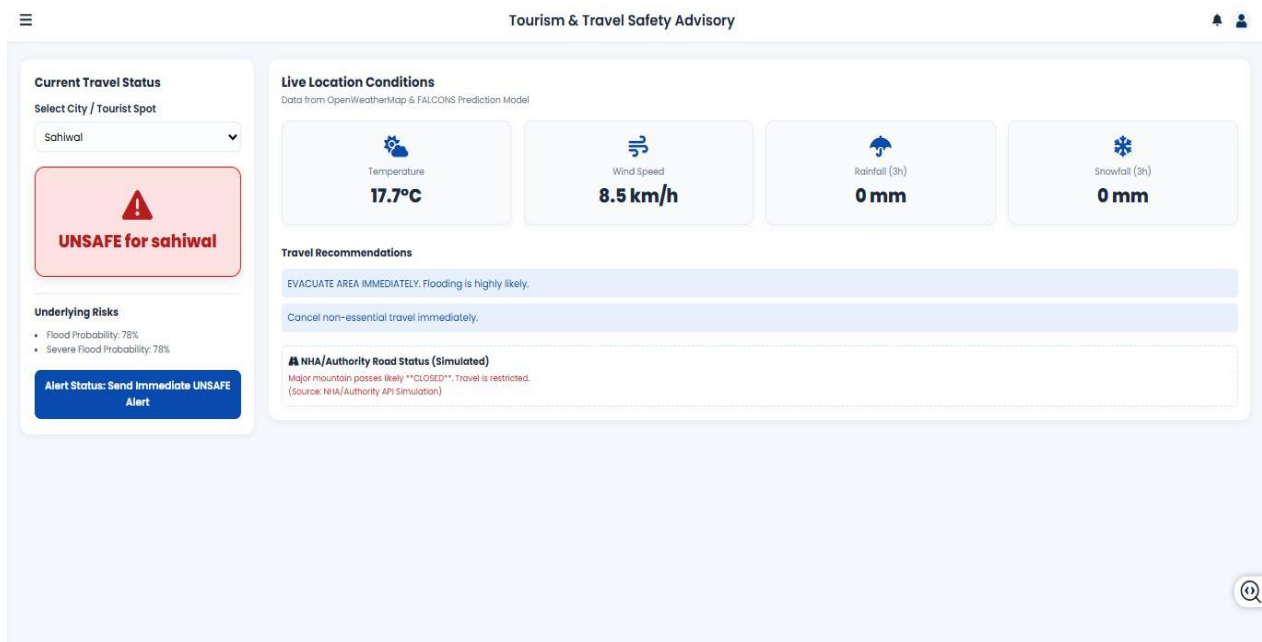


Figure 5.3-5 Tourism safety

5.3.6 Alert Management Screen

Figure 5.3.6 illustrates the Alert Management Screen, which is responsible for displaying all system-generated alerts. Alerts are categorized based on severity levels and geographical areas. The system ensures that high-risk alerts are prominently highlighted for immediate attention. Users receive timely notifications to stay informed about potential dangers. The interface is designed to present alerts in a clear and structured format. This screen plays a crucial role in emergency communication and response.

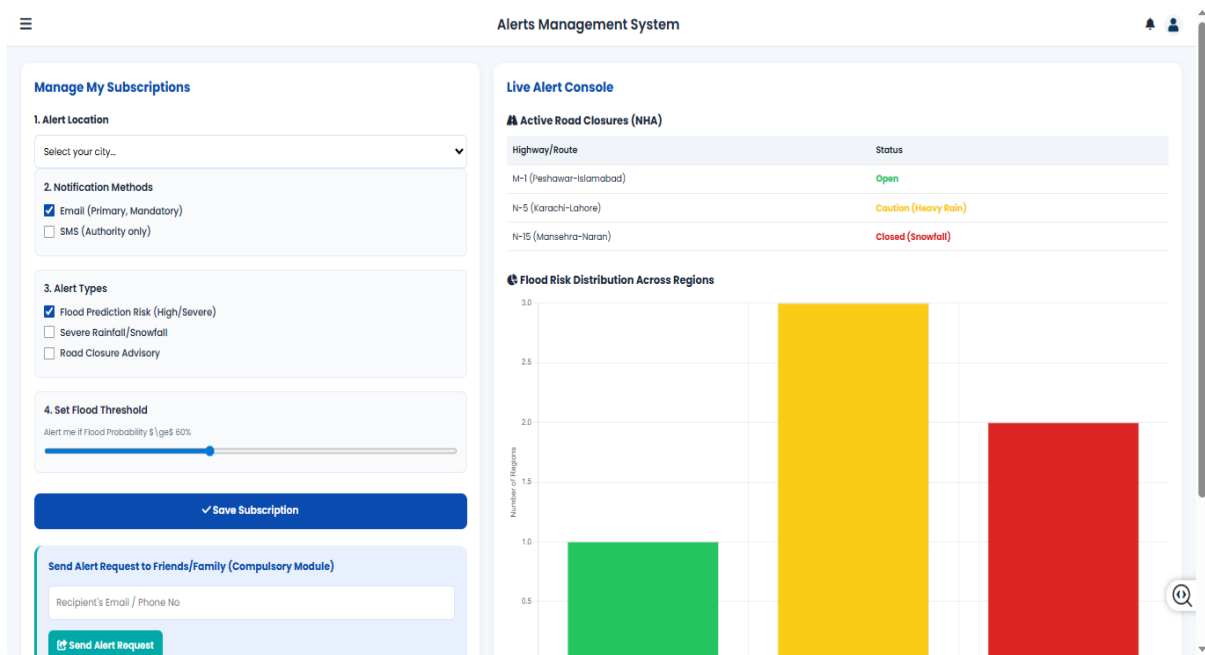


Figure 5.3-6 Alert Management

5.3.7 Rescue Reporting Screen

Figure 5.3.7 illustrates the Rescue Reporting Screen, which allows users to report flood incidents or request emergency assistance. Users can provide details such as location, description, and severity of the situation. The interface is designed for quick and simple data entry, especially during emergency scenarios. Once submitted, the information is stored in the system and made available for administrative action. This feature enhances coordination between users and response teams. It is an essential component for effective disaster management.

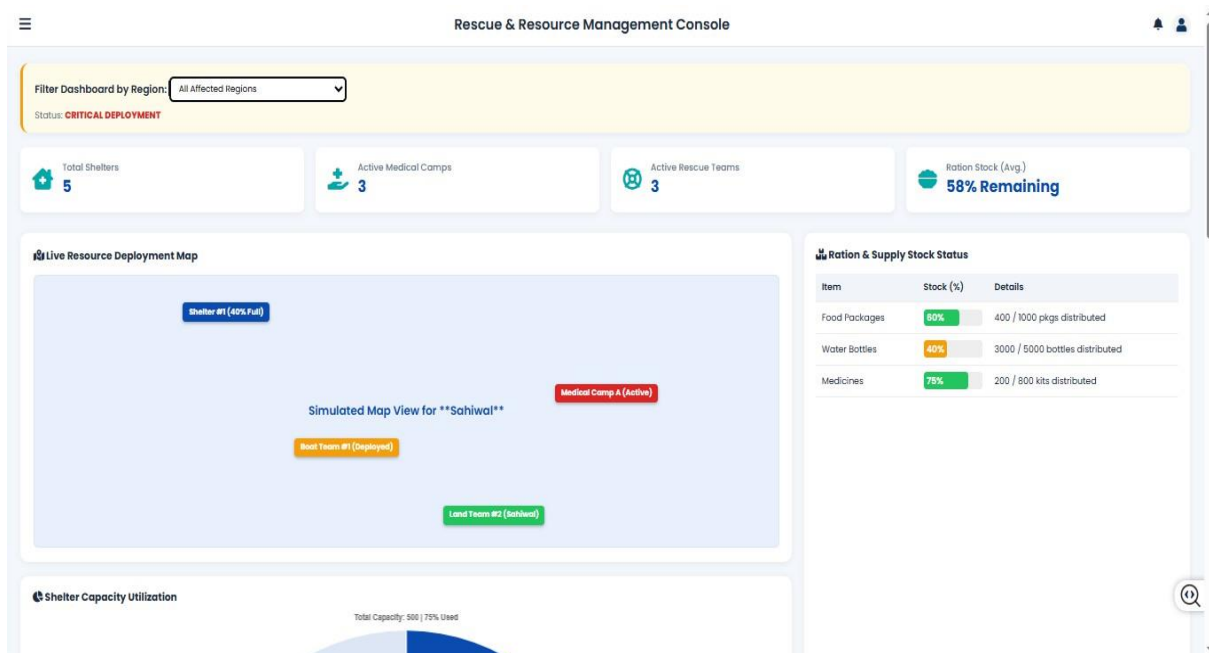
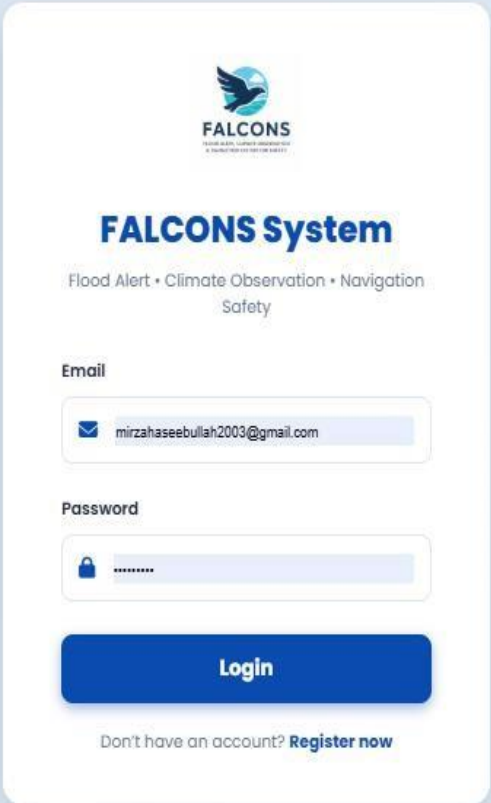


Figure 5.3-7 Rescue Report

5.3.8 Login screen

Figure 5.3.8 illustrates the Login Screen, which provides secure access to the FALCONS platform for registered users and administrators. It includes fields for entering user credentials such as email and password. The system validates input data to ensure secure authentication. Error-handling mechanisms are implemented to guide users in case of incorrect credentials. The design is simple and user-friendly to support easy access. This screen ensures that only authorized users can access system features and data.

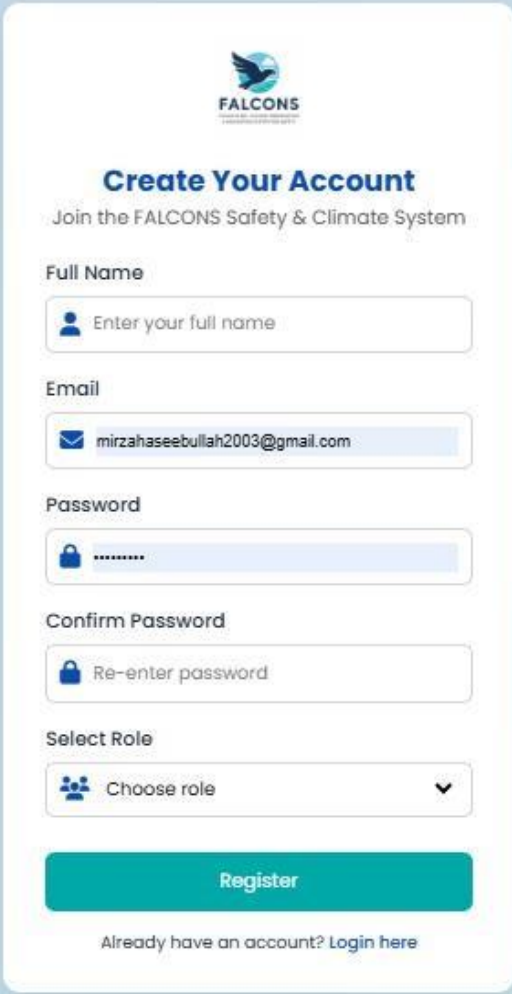


The login screen for the FALCONS System is displayed on a light blue background. It features a white card with the FALCONS logo at the top, which includes a blue eagle icon and the text "FALCONS" and "FLOOD ALERT • CLIMATE OBSERVATION • NAVIGATION • SAFETY". Below the logo, the text "FALCONS System" is prominently displayed in blue. Underneath, the system's functions are listed: "Flood Alert • Climate Observation • Navigation • Safety". The login form consists of two input fields: "Email" and "Password". The "Email" field contains the text "mirzahaseebullah2003@gmail.com". The "Password" field is masked with asterisks. A blue "Login" button is positioned below the password field. At the bottom of the card, there is a link that says "Don't have an account? Register now".

Figure 5.3-8 Login Screen

5.3.9 Create account Screen

Figure 5.3.9 illustrates the Create Account Screen of the FALCONS platform, which enables new users to create their accounts and gain access to system features. This screen includes input fields for user details such as name, email, password, and password confirmation. The system performs validation to ensure that all required information is entered correctly and securely. In case of invalid or missing input, appropriate error messages are displayed to guide the user. The interface is designed to be simple, clear, and easy to use, allowing users to complete the account creation process quickly.



The screenshot displays the 'Create Your Account' interface for the FALCONS platform. At the top, the FALCONS logo is visible, followed by the title 'Create Your Account' and the subtitle 'Join the FALCONS Safety & Climate System'. The form contains the following elements:

- Full Name:** A text input field with a user icon and the placeholder text 'Enter your full name'.
- Email:** A text input field with an email icon and the placeholder text 'mirzahaseebullah2003@gmail.com'.
- Password:** A password input field with a lock icon and masked characters '*****'.
- Confirm Password:** A text input field with a lock icon and the placeholder text 'Re-enter password'.
- Select Role:** A dropdown menu with a group of people icon and the placeholder text 'Choose role'.
- Register:** A prominent teal button.
- Login here:** A link for existing users, preceded by the text 'Already have an account?'.

Figure 5.3-9 Create Account page

Chapter 6

TESTING AND EVALUATION

6. Testing and Evaluation

Testing is a critical phase in the development of the lifecycle of the **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)** platform, ensuring that the system operates accurately, reliably, and efficiently under different conditions. Since the system deals with real-time environmental data, flood prediction logic, and emergency response features, a comprehensive testing strategy is essential to maintain system integrity and user trust [5].

The primary objective of testing in FALCONS is to validate the correctness of flood prediction results, ensure proper functioning of alert mechanisms, and verify the reliability of rescue reporting and user interaction modules. Special attention is given to validating data processing from datasets, ensuring that predictions reflect realistic environmental conditions such as rainfall, humidity, and temperature.

Testing is conducted at multiple levels, including unit testing, integration testing, functional testing, and system testing. Each module is evaluated individually and then tested collectively to ensure smooth communication between components such as the prediction engine, alert system, and database.

Security testing is also performed to ensure that user data stored in **Supabase** is protected and that unauthorized access is prevented through proper authentication mechanisms. Performance testing ensures that the system can handle multiple users simultaneously without delays, especially during critical situations [7].

Through a structured and continuous testing approach, the FALCONS system ensures a stable, accurate, and responsive platform capable of supporting disaster management effectively.

6.1 Manual Testing

Manual testing plays a vital role in validating the usability and functionality of the FALCONS platform. Testers interact with the system as real users to evaluate different features such as flood prediction, weather monitoring, alert notifications, and rescue reporting.

During this process, testers verify that users can easily navigate through the platform, select cities, view predictions, and submit reports without errors. Special attention is given to ensuring that the system responds correctly under different scenarios, such as invalid inputs or missing data.

Manual testing also helps identify user interface issues, navigation problems, and inconsistencies that automated testing might not detect. This ensures that the system provides a smooth and user-friendly experience, especially during emergency situations.

System Testing

System testing is performed to evaluate the FALCONS platform as a complete and integrated system. This phase ensures that all modules, including flood prediction, weather data visualization, alert generation, and rescue reporting, work together seamlessly.

The objective is to verify that the system meets all functional and non-functional requirements. It checks whether predictions are generated correctly, alerts are triggered appropriately, and reports are stored and processed efficiently.

System testing also evaluates real-world scenarios, such as high rainfall conditions or multiple user requests, to ensure system stability. This phase guarantees that the platform is ready for deployment and can operate reliably during critical situations.

Unit Testing

Unit testing focuses on validating individual components of the FALCONS system independently. Each function, such as user authentication, data retrieval from Supabase, and prediction logic, is tested in isolation.

For example, unit tests verify whether the flood prediction algorithm correctly processes input values and generates appropriate risk levels. Similarly, authentication modules are tested to ensure secure login and session management.

This approach helps identify errors early in the development process, making debugging easier and improving overall system quality.

6.1.1 Unit Testing

Unit Testing 1: Login as Admin

Testing Objective: Ensure the admin login form functions correctly.

Table 6-1: Admin Login

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Admin Login by entering username and password	Username: admin@offplatform.com Password: adminSecure@2025	Admin successfully logs into the system	Pass

Unit Testing 2: Login as a User

Testing Objective: Ensure the user login and registration system works as expected.

Table 6.2: User Login

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-002	User Login by entering username and password	Username: user1@off.com Password: userPass@123	User successfully logs into the system	Pass

Unit Testing 3: Logout as Admin

Testing Objective: Ensure admin can log out successfully.

Table 6.3: Admin Logout

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-003	Click the logout button	Logout	Admin successfully exits the system	Pass

Unit Testing 4: Logout as User

Testing Objective: Ensure user can log out successfully.

Table 6.4: User Logout

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-004	Click the logout button	Logout	User successfully exits the system	Pass

6.2 Functional Testing

6.2.1 Functional Testing 1: Login with Different Roles

Objective: Ensure role-based access and navigation load correctly.

Table 6.5: Login as Different Roles

No.	Test Case	Attribute & Value	Expected Result	Result
1	Login as Admin	Username: admin@offplatform.com Password: adminSecure@2025	Admin dashboard loads with admin navigation	Pass
2	Login as User	Username: customer01@off.com Password: tryon2025	User interface loads with prediction features	Pass
3	Access Alerts	Username: manager@off.com Password: stockAdmin2025	Access Alerts	Pass

6.3 Integration Testing

Objective: Ensure modules such as login, try-on, product browsing, and cart functionalities work cohesively.

Table 6.6: Integration Testing

No.	Test Case	Attribute & Value	Expected Result	Result
1	Login and check the prediction	Username: user1@off.com Password: userPass@123	Flood prediction displayed	Pass
2	View weather data	Action: Select city weather	Weather data shown correctly	Pass
3	Generate alert	Action: Click High-risk condition triggered	Alert notification displayed	Pass
4	Submit rescue report	Action: User submits report form	Report stored in database	Pass
5	View map data	Action: Click Open flood map	Map loads with correct visualization	Pass

Chapter 7

CONCLUSION AND FUTURE WORK

7. Conclusion

The **FALCONS (Flood Alert, Climate Observation & Navigation System for Safety)** platform presents a comprehensive and intelligent solution for managing flood-related risks and enhancing public safety through technology. The system is designed as a web-based application that integrates multiple critical functionalities, including flood prediction, real-time weather monitoring, alert generation, tourism safety guidance, and rescue reporting [4].

At the core of the platform lies a data-driven flood prediction mechanism that analyzes environmental parameters such as rainfall, humidity, and temperature to estimate potential flood risks. This allows users to receive early warnings and take preventive actions. The integration of real-time data visualization and user-friendly dashboards ensures that complex environmental information is presented simply and understandably [2].

Unlike traditional systems that rely heavily on manual monitoring or isolated tools, FALCONS provides a unified platform where users, administrators, and emergency responders can interact efficiently. The use of **Supabase** as a backend solution enables secure data storage, authentication, and real-time updates without the need for complex server management. This makes the system lightweight, scalable, and easy to maintain.

The platform also emphasizes user interaction through features such as rescue reporting, which allows individuals to report emergencies quickly, improving response coordination. Additionally, alert management ensures that users are informed about critical situations promptly, enhancing disaster preparedness [1].

7.1 Future Work

While the current implementation of FALCONS provides a strong foundation for flood prediction and disaster management, several enhancements can further improve its performance, scalability, and user experience. Future work can focus on the following areas:

7.1.1 Enhanced AI-Based Try-On Accuracy

Future improvements can focus on enhancing the flood prediction model by incorporating advanced machine learning algorithms. By using larger datasets and additional environmental factors such as river water levels and satellite data, the system can generate more accurate and reliable predictions. Continuous model training can further improve prediction performance over time [2].

7.1.2 Smart Alert System

The alert system can be upgraded to provide more intelligent and personalized notifications. Location-based alerts can ensure that users only receive relevant warnings for their specific areas. Integration with SMS and mobile notifications can also improve reach, especially during emergencies when internet access may be limited [5].

7.1.3 Mobile Application Development

To improve accessibility, a dedicated mobile application for Android and iOS can be developed. This would allow users to receive instant alerts, check flood predictions, and submit rescue reports directly from their smartphones. Mobile integration will enhance user engagement and ensure faster communication during disasters [5].

7.1.4 Integration with IoT and Sensors

Future enhancements of the FALCONS platform can significantly benefit from the integration of Internet of Things (IoT) devices and environmental sensing technologies. By incorporating sensors such as water level detectors, rainfall gauges, humidity sensors, and river flow monitoring devices, the system can collect highly accurate real-time data directly from the environment. This data can be continuously transmitted to the system and stored using Supabase, enabling dynamic updates in flood prediction models. Unlike static datasets, real-time sensor data allows the system to respond instantly to changing environmental conditions, thereby improving prediction accuracy and timeliness. Furthermore, IoT integration can support automated alert generation when critical thresholds are exceeded, reducing dependency on manual monitoring [9].

7.1.5 Advanced Data Visualization

Advanced data visualization is an essential area for future improvement in the FALCONS platform, as it directly impacts how effectively users interpret critical information. By integrating interactive visual components such as dynamic maps, heatmaps, and trend-based graphs, the system can present complex environmental data in a more intuitive and meaningful way. For instance, flood-prone areas can be highlighted using color-coded risk zones, allowing users to quickly identify high-risk regions. Time-series graphs can illustrate rainfall patterns and flood trends over specific periods, helping users and authorities understand how conditions are evolving. Additionally, integrating map-based visualization tools can allow users to select specific locations and view localized predictions and alerts. These enhancements not only improve user experience but also support better decision-making

during emergencies. For administrators and disaster management authorities, detailed visual analytics can provide insights into patterns, system performance, and affected regions. Overall, advanced visualization transforms raw data into actionable insights, making the FALCONS platform more effective, informative, and user-centric [3].

7.1.6 Admin Analytics and Monitoring

The admin panel can be enhanced with advanced analytics tools to monitor system performance, user activity, and flood trends. This will help administrators make data-driven decisions and improve system efficiency.

7.1.7 Multi-Language and Accessibility Support

To make the platform accessible to a wider audience, future development can include multi-language support and accessibility features. This will ensure usability for people from different regions and those with disabilities.

7.1.8 Tourism Safety Enhancements

Future improvements can expand the tourism safety module by including travel advisories, safe route navigation, and real-time risk assessments. This will help travelers make safer decisions during adverse conditions.

Chapter 8

REFERENCE

8. References

- [1] B. Mosavi, P. Ozturk, and K. Chau, “Flood prediction using machine learning models: Literature review,” *Water*, vol. 10, no. 11, pp. 1–40, 2018, doi: 10.3390/w10111536.
- [2] S. K. Jain, V. P. Singh, and R. K. Jain, “Flood forecasting using artificial neural networks: A review,” *Journal of Hydrology*, vol. 539, pp. 310–329, 2016, doi: 10.1016/j.jhydrol.2016.05.037.
- [3] Y. Liu, X. Zhou, and Y. Zhu, “Application of deep learning in flood prediction: A review,” *Environmental Modelling & Software*, vol. 117, pp. 163–177, 2019, doi: 10.1016/j.envsoft.2019.03.017.
- [4] A. K. Lohani, N. K. Goel, and K. K. Bhatia, “Improving real-time flood forecasting using data-driven techniques,” *Water Resources Management*, vol. 28, no. 2, pp. 365–382, 2014.
- [5] H. Wu, Q. Chen, and J. Chen, “Real-time flood monitoring and early warning system based on IoT,” *IEEE Access*, vol. 8, pp. 123–135, 2020, doi: 10.1109/ACCESS.2020.2961234.
- [6] M. A. Islam, M. S. Rahman, and M. A. Rahman, “Flood hazard mapping using GIS and remote sensing techniques,” *Natural Hazards*, vol. 75, no. 1, pp. 1–20, 2015.
- [7] J. Smith and L. Brown, “Big data analytics for disaster management systems,” *International Journal of Disaster Risk Reduction*, vol. 35, 2019.
- [8] P. K. Tiwari and R. Mishra, “Climate change impact on flood frequency and severity,” *Environmental Science & Policy*, vol. 100, pp. 27–36, 2019.
- [9] Supabase Inc., “Supabase Documentation,” 2024.
- [10] World Meteorological Organization, “Flood forecasting and early warning systems,” WMO Report, 2021.